

**PANEURÓPSKA VYSOKÁ ŠKOLA
FAKULTA INFORMATIKY**

FI-XXXXXX-XXXXX

**RENTME: WEBOVÁ PLATFORMA PRE INZERCIU SLUŽIEB A
PRENÁJMU**

Bakalárska práca

2026

Martin Muha

**PANEURÓPSKA VYSOKÁ ŠKOLA
FAKULTA INFORMATIKY**

**RENTME: WEBOVÁ PLATFORMA PRE INZERCIU SLUŽIEB A
PRENÁJMU**

Bakalárska práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Ústav aplikovanej informatiky
Školiteľ:	Mgr. Michal Drobný

Bratislava 2026
Martin Muha



PANEURÓPSKA VYSOKÁ ŠKOLA

Fakulta informatiky

ZADANIE PRÁCE

Meno a priezvisko študenta:

Evidenčné číslo práce:

Študijný odbor:

Študijný program:

Forma a metóda štúdia:

Vedúci práce:

Ústav/katedra:

Dátum zadania práce:

Názov:

Anotácia: Cieľom práce je vytvorenie komplexného systému pre navigáciu v interiéroch s napojením na rozličné webové zdroje s možnosťou ich editovania a návrh shaderov pre ich zobrazenie. Súčasťou práce bude analýza použitia v reálnom nasadení vrátane implementácie a zobrazovania na rozličných dotykových zariadeniach.

.....
RNDr. Ján Lacko, PhD.
vedúci práce

.....
Ing. Juraj Štefanovič, PhD.
vedúci ústavu

Čestné vyhlásenie

Čestne vyhlasujem, že som záverečnú prácu vypracoval samostatne a uviedol som všetku použitú literatúru.

.....

Pod'akovanie

Pri vypracovaní tejto bakalárskej práce by som sa rád pod'akoval Mgr. Michalovi Drobnému za jeho odborné vedenie, cenné rady a konzultácie, ktoré mi významne pomohli pri spracovaní danej témy.

Martin Muha

ABSTRAKT

MARTIN MUHA: RentMe: Webová platforma pre inzerciu služieb a prenájmu. Bakalárska práca. Paneurópska vysoká škola. Fakulta informatiky. Školiteľ: Mgr. Michal Drobný. Bratislava. FI PEVŠ, 2026, 59 s.

Táto práca sa zameriava na návrh a analýzu webovej aplikácie RentMe, ktorá predstavuje inovatívnu digitálnu platformu pre prenájom vecí a poskytovanie služieb v rámci jedného integrovaného systému. Aplikácia kombinuje princípy zdieľanej ekonomiky a gig ekonomiky, pričom umožňuje používateľom nielen prenajímať a vyhľadávať rôzne druhy majetku, ale zároveň ponúkať a získavať pracovné príležitosti a služby prostredníctvom inzertného systému. Cieľom aplikácie RentMe je zjednodušiť proces sprostredkovania prenájmov a služieb, zvýšiť dostupnosť lokálnych pracovných príležitostí a podporiť efektívne využívanie existujúcich zdrojov. Práca sa venuje analýze konkurenčných platforiem, návrhu funkcionalít aplikácie, technologickej architektúre a identifikácii hlavných prínosov navrhovaného riešenia. Výsledkom je koncept univerzálnej webovej platformy, ktorá prepája viacero trhových segmentov do jedného ekosystému a reaguje na aktuálne trendy digitalizácie, flexibility práce a zdieľania zdrojov.

Kľúčové slová: webová aplikácia, prenájom, zdieľaná ekonomika, gig ekonomika, digitálna platforma

ABSTRACT

MARTIN MUHA: RentMe – Web Platform for Advertising Services and Rental. Bachelor thesis. Pan-European University. Faculty of Informatics. Thesis supervisor: Mgr. Michal Drobný. Bratislava. FI PEVŠ, 2026, 59 p.

This thesis focuses on the design and analysis of RentMe, a web application that represents an innovative digital platform for item rental and service provision within a single integrated system. The application combines principles of the sharing economy and gig economy, enabling users not only to rent and search for various types of assets, but also to offer and obtain employment opportunities and services through a classified advertisement system. The objective of the RentMe application is to streamline the process of mediating rentals and services, enhance the availability of local employment opportunities, and promote efficient utilization of existing resources. This work addresses the analysis of competing platforms, the design of application functionalities, technological architecture, and the identification of the main benefits of the proposed solution. The result is a concept of a universal web platform that interconnects multiple market segments into one ecosystem and responds to current trends in digitalization, work flexibility, and resource sharing.

Keywords: web application, rental, sharing economy, gig economy, digital platform

Obsah

Zoznam ilustrácií	9
Zoznam tabuliek	10
Zoznam skratiek a značiek	11
1 ÚVOD	13
1.1 MOTIVÁCIA A VÝZNAM INZERTNÝCH PLATFORIEM V SÚČASNOSTI	13
1.1.1 Digitalizácia, dostupnosť a sieťový efekt	13
1.1.2 Lokalizácia a geografický kontext	13
1.1.3 Jednoduchosť a používateľská prívetivosť	13
1.1.4 Dôvera, bezpečnosť a transparentnosť	14
1.2 DEFINÍCIA PROBLÉMU (CENTRALIZÁCIA SLUŽIEB A PRENÁJMU)	14
1.3 CIELE BAKALÁRSKEJ PRÁCE	15
2 SÚČASNÝ STAV RIEŠENEJ PROBLEMATIKY	16
2.1 ANALÝZA KONKURENČNÝCH PLATFORIEM A SYSTÉMOV	16
2.1.1 Airbnb.com	16
2.1.2 OLX/Bazoš	16
2.1.3 Booking.com	17
2.1.4 Fiverr.com	17
2.1.5 Turo.com	18
2.2 EKONOMICKÉ MODELY MODERNEJ INZERCIE	18
2.2.1 Fenomén Micro-gig economy	18
2.2.2 Model Peer-to-Peer (P2P) a zdieľaná ekonomika	19
2.2.3 Slovenský trh a lokálna príležitosť	20
2.3 TECHNOLOGICKÉ TRENDY A PRIDANÁ HODNOTA RIEŠENIA	20
3 ANALÝZA A NÁVRH SYSTÉMU	21
3.1 FUNKČNÉ POŽIADAVKY	21
3.1.1 Návštevník	21
3.1.2 Používateľ	21
3.1.3 Administrátor	23
3.2 NEFUNKČNÉ POŽIADAVKY	24
3.2.1 Bezpečnosť	24
3.2.2 Škálovateľnosť	25
3.2.3 Výkon	25
3.3 NÁVRH ARCHITEKTÚRY	25
3.3.1 Štruktúra monorepa	26
3.3.2 Model klient–server	26
3.3.3 Dátový tok	27
3.4 DATABÁZOVÝ MODEL	27

3.4.1	Hlavné entity systému a ich význam	27
3.4.2	Entitno-relačný diagram (ERD)	29
3.5	REST API A JWT	31
3.5.1	Návrh endpointov a smerovania	31
3.5.2	Autentifikačný mechanizmus (JWT)	32
3.5.3	Validácia a transformácia údajov	32
3.5.4	Spracovanie chýb (Error Handling)	33
4	METODIKA A POUŽITÉ TECHNOLOGIE	34
4.1	FULL-STACK VÝVOJ V JAZYKU TYPESCRIPT	34
4.1.1	Typová bezpečnosť a eliminácia chýb	34
4.1.2	Zdieľanie kódu v monorepe	34
4.1.3	Lepšia čitateľnosť, dokumentácia a Developer Experience (DX)	35
4.1.4	Moderné funkcie, dekorátory a kompatibilita	35
4.2	BACKEND: FRAMEWORK NESTJS A MODULÁRNA ARCHITEKTÚRA	36
4.2.1	Modulárna štruktúra a organizácia kódu	36
4.2.2	Dependency Injection (Injektovanie závislostí)	37
4.2.3	Middleware a Guardy pre bezpečnosť	37
4.2.4	Integrácia s Prisma ORM	37
4.3	FRONTEND: NEXT.JS (APP ROUTER), REACT A TAILWIND CSS	38
4.3.1	Next.js a architektúra App Router	38
4.3.2	Deklaratívne UI s knižnicou React	38
4.3.3	Styling pomocou Tailwind CSS	39
4.3.4	Integrácia s mapovými podkladmi (Leaflet)	40
4.4	PERZISTENCIA DÁT: POSTGRESQL A PRISMA ORM	40
4.4.1	Relačná databáza PostgreSQL	40
4.4.2	Prisma ORM (Object-Relational Mapping)	41
4.4.3	Prospech pre Monorepo architektúru	42
4.5	VOL'BA TECHNOLOGIÍ: NODE.JS EKOSYSTÉM VERZUS PHP	42
4.5.1	Asynchrónny model spracovania požiadaviek	42
4.5.2	Jednotný jazyk naprieč celým stackom	42
4.5.3	Server-Side Rendering a SEO	43
4.5.4	Ekosystém, nástroje a Developer Experience	43
4.5.5	Zhrnutie porovnania	43
4.6	NÁSTROJE PRE SPRÁVU KÓDU A KONTAJNERIZÁCIU (GIT, DOCKER COM- POSE)	44
4.6.1	Verzovanie kódu pomocou systému Git	44
4.6.2	Kontajnerizácia pomocou technológie Docker	44
4.6.3	Orchestrácia služieb cez Docker Compose	45
4.6.4	Prepojenie s monorepom a optimalizácia obrazov	45
5	REALIZÁCIA A IMPLEMENTÁCIA SYSTÉMU	46
5.1	ŠTRUKTÚRA MONOREPA A ZDIEĽANIE TYPOV	46

5.1.1	Organizácia adresárovej štruktúry	46
5.1.2	Implementácia packages/shared a zdieľanie typov	46
5.1.3	Mechanizmus npm workspaces	47
5.2	REALIZÁCIA BACKENDOVÝCH MODULOV	47
5.2.1	Modul Auth: Autentifikácia a autorizácia	47
5.2.2	Modul Ads: Správa inzerátov a médií	48
5.2.3	Modul Categories a dynamické filtre	49
5.2.4	Modul Search: Full-textové vyhľadávanie a analytika	50
5.3	VÝVOJ KLIENSKYCH APLIKÁCIÍ	50
5.3.1	Verejná platforma (vyhľadávanie, mapa, blog)	50
5.3.2	Používateľská zóna (správa inzerátov a profilu)	51
5.3.3	Administračný panel (CMS, vizuálny editor GrapesJS, štatistiky)	51
5.4	BEZPEČNOSTNÉ ASPEKTY: HASHOVANIE HESIEL (BCRYPT), VALIDÁCIA VSTUPOV (CLASS-VALIDATOR) A OCHRANA ÚČTOV (BANOVANIE)	52
5.5	SYSTÉM DYNAMICKÝCH FILTROV A KATEGÓRIÍ	52
6	TESTOVANIE A VÝSLEDKY	54
6.1	OVERENIE FUNKČNOSTI API (SWAGGER DOKUMENTÁCIA)	54
6.2	ANALYTIKA A MONITORING (CLICKEVENT TRACKING A SEGMENTÁCIA POUŽÍVATEĽOV)	54
6.3	VYHODNOTENIE POUŽÍVATEĽSKÉHO ROZHRAŇA A RESPONZIVITY	55
7	DISKUSIA A PRÍNOSY PRÁCE	56
7.1	SPRÁVA OBSAHU BEZ NUTNOSTI REDEPLOYU (SITEMENU, SITECONFIG)	56
7.2	MOŽNOSTI ĎALŠIEHO ROZŠÍRENIA (PLATOBNÁ BRÁNA, EXTERNÉ ÚLOŽISKO MÉDIÍ)	56
8	ZÁVER	58
8.1	ZHRNUTIE DOSIAHNUTÝCH VÝSLEDKOV	58
8.2	NAPLNENIE STANOVENÝCH CIEĽOV	58
	Použitá literatúra	59

Zoznam ilustrácií

1	Diagram použitia (Use Case) – funkčné požiadavky a aktéri systému	22
2	Tok zápisu dát po validácii: kontrolér, aplikačná služba, Prisma ORM a databáza PostgreSQL	28
3	Entitno-relačný diagram (ERD) databázy – jadrové entity, podporné tabuľky a ich relácie	30

Zoznam tabuliek

1	Prehľad hlavných modulov API a základných ciest	32
2	Porovnanie PHP (Laravel) a Node.js/TypeScript ekosystému	43

Zoznam skratiek a značiek

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CMS	Content Management System
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DI	Dependency Injection
DTO	Data Transfer Object
ERD	Entity-Relationship Diagram
GPS	Global Positioning System
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JS	JavaScript
JSON	JavaScript Object Notation
JSONB	JSON Binary (PostgreSQL)
JWT	JSON Web Token
MVC	Model-View-Controller
ORM	Object-Relational Mapping
P2P	Peer-to-Peer
REST	Representational State Transfer
RSC	React Server Components
SEO	Search Engine Optimization
SQL	Structured Query Language
SRP	Single Responsibility Principle
SSR	Server-Side Rendering
TS	TypeScript
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

XSS

Cross-Site Scripting

1 ÚVOD

1.1 MOTIVÁCIA A VÝZNAM INZERTNÝCH PLATFORMIEM V SÚČASNOSTI

V súčasnej digitálnej ére prechádza spôsob, akým spotrebitelia vyhľadávajú služby a prenájmy, zásadnou transformáciou (Vallas a Schor 2020). Tradičné formy inzercie sú nahrádzané interaktívnymi webovými platformami, ktoré umožňujú okamžitý prístup k informáciám v reálnom čase. Hlavnou motiváciou pre vývoj modernej inzertnej platformy je potreba konsolidovať rozptýlenú ponuku do jedného transparentného a bezpečného ekosystému.

1.1.1 Digitalizácia, dostupnosť a sieťový efekt

Moderný používateľ očakáva vysokú mieru flexibility a rýchlosti pri hľadaní odborných služieb či prenájmu. Kľúčovým faktorom úspechu je budovanie mena aplikácie a zvyšovanie jej návštevnosti, čím vzniká takzvaný sieťový efekt – s rastúcim počtom inzerátov a používateľov sa zvyšuje celkový dopyt po ponukách a hodnota platformy pre celú komunitu (Rochet a Tirole 2003). Digitalizácia týchto procesov odbúrava administratívne bariéry a umožňuje efektívnejšie párovanie dopytu s ponukou.

1.1.2 Lokalizácia a geografický kontext

Pri službách a prenájmoch zohráva geografická poloha kritickú úlohu. Implementácia pokročilých mapových rozhraní a lokalizačných služieb (v projekte riešené cez knižnicu Leaflet) umožňuje používateľom vizualizovať dostupnosť ponúk vo svojom bezprostrednom okolí (Obe a Hsu 2017). Tento prístup zvyšuje používateľský komfort a znižuje logistické náklady, pričom geografické vyhľadávanie sa stáva štandardom odlišujúcim profesionálne riešenia od jednoduchých textových zoznamov.

1.1.3 Jednoduchosť a používateľská prívetivosť

Okrem funkčnej vybavenosti je kľúčovým prvkom významu platformy jej jednoduchosť. Intuitívne používateľské rozhranie zabezpečuje, že aplikácia je prístupná širokému spektru osôb bez ohľadu na ich technické zručnosti (Riva 2022). Práve nízka bariéra vstupu a rýchly proces publikácie inzerátu priamo podmieňujú ochotu používateľov opakovane sa do systému vracieť. Výskumy v oblasti UX (User Experience) opakovane potvrdzujú, že každé zbytočné kliknutie alebo krok vo formulári znižuje mieru dokončenia o desiatky percent – preto som zvolil viacstupňového sprievodcu (wizard) namiesto dlhého jednostranového formulára.

1.1.4 Dôvera, bezpečnosť a transparentnosť

Jednou z najväčších výziev online inzercie je eliminácia podvodného správania (Hoffman 2020). Význam platformy spočíva v schopnosti garantovať bezpečnosť prostredníctvom aktívnej moderácie, systémov na nahlasovanie podozrivých aktivít (modul `reports`) a prísnych pravidiel pre schvaľovanie inzerátov administrátorom. Tieto prvky, spolu s prehľadnou históriou správ, budujú nevyhnutnú dôveru medzi subjektmi v digitálnom priestore (Sundarajan 2014).

1.2 DEFINÍCIA PROBLÉMU (CENTRALIZÁCIA SLUŽIEB A PRENÁJMU)

Súčasný digitálny trh so službami a prenájmom na Slovensku je príliš rozdrobený, čo komplikuje kontakt medzi tými, ktorí službu ponúkajú, a tými, ktorí ju hľadajú. Hlavným problémom je chýbajúca univerzálna platforma, ktorá by dokázala spojiť rôzne profesie a typy prenájmu na jednom mieste. Používatelia sú dnes nútení hľadať riešenia cez viaceré, často špecializované alebo neformálne kanály, ktoré však nespĺňajú všetky ich potreby.

Dobрым príkladom špecializácie je portál `pretlak.com`. Ponúka síce moderné prostredie, ale zameriava sa takmer výhradne na IT, marketing a kreatívny priemysel. Podobne sú na tom aj skupiny na sociálnych sieťach, napríklad skupina `Programátori`, ktorá slúži len úzkemu okruhu odborníkov na vývoj. Ostatné remeslá a profesie tak často končia na portáli `bazos.sk`. Ten je však v prvom rade určený na predaj tovaru, nie na správu služieb. Navyše, jeho vizuálna stránka a ovládanie cielia skôr na staršie generácie, čo pre mladších používateľov, zvyknutých na moderné a rýchle aplikácie, predstavuje technologickú prekážku.

Ďalšou výraznou slabinou je chýbajúca štruktúra a slabé možnosti filtrovania. Na sociálnych sieťach sa príspevky radia za sebou podľa času pridania, kvôli čomu zaujímavé ponuky rýchlo zapadnú. Keďže chýba podrobné vyhľadávanie, používateľ musí manuálne prechádzať množstvo inzerátov, ktoré ho nezaujímajú. Problémom je aj chýbajúci geografický prehľad. Bez mapy a presného určenia polohy je veľmi ťažké a zdĺhavé nájsť niekoho, kto poskytuje služby priamo v okolí používateľa.

Z pohľadu bezpečnosti sú sociálne siete a fóra rizikové, pretože im chýbajú nástroje na kontrolu obsahu alebo overovanie osôb. Ak niekto poruší pravidlá, administrátor nemá k dispozícii systém, ktorý by okamžite zasiahol, čo vytvára priestor pre podvody. Prevádzkovatelia týchto skupín navyše nevedia sledovať, o čo majú ľudia skutočný záujem, pretože nemajú prístup k analytickým dátam. Moja práca na tieto nedostatky reaguje vytvorením centralizovanej platformy, ktorá spája všetky potrebné funkcie do jedného prehľadného systému s jasnými pravidlami a jednoduchým ovládaním.

1.3 CIELE BAKALÁRSKEJ PRÁCE

Hlavným cieľom tejto bakalárskej práce je navrhnúť a vytvoriť modernú webovú platformu, ktorá slúži na uverejňovanie a správu inzerátov v oblasti služieb a prenájmu. Celý systém je navrhnutý tak, aby na jednom mieste prepojil bežných používateľov, poskytovateľov služieb a administrátorov, čím nahrádza súčasné neprehľadné hľadanie na sociálnych sieťach alebo všeobecných inzertných weboch.

Pre dosiahnutie tohto hlavného cieľa som si stanovil niekoľko konkrétnych úloh:

Prvou úlohou je vytvorenie stabilného a bezpečného technického zázemia (backendu) pomocou frameworku `NestJS`. Tento systém musí zvládnuť bezpečnú registráciu a prihlasovanie používateľov cez `JWT` tokeny, správne ukladať dáta do databázy `PostgreSQL` a zabezpečiť, aby sa k citlivým údajom nedostal nikto nepovoláný. Dôležitým bodom je aj kontrola vstupných údajov, aby sa predišlo chybám alebo útokom na aplikáciu.

Druhou úlohou je vývoj troch samostatných webových aplikácií pomocou technológie `Next.js`, ktoré sú prispôbosené konkrétnym skupinám ľudí. Verejná časť umožní návštevníkom rýchlo vyhľadávať inzeráty aj pomocou interaktívnej mapy, používateľská zóna dovolí registrovaným členom spravovať svoje ponuky a správy, a administratívny panel poskytne nástroje na schvaľovanie obsahu a sledovanie štatistík. Pri tvorbe vzhľadu sa zameriavam na jednoduchosť a moderný dizajn, aby bola aplikácia príjemná pre mladšiu generáciu, no zároveň zrozumiteľná pre každého.

Posledným dôležitým cieľom je implementácia analytických nástrojov a funkcií na moderovanie obsahu. Systém bude automaticky sledovať kliknutia na inzeráty, čo správcovi pomôže pochopiť, o ktoré kategórie je najväčší záujem. Zároveň bude platforma obsahovať funkcie na nahlasovanie nevhodných inzerátov a možnosť zablokovat' problémových používateľov, čím sa zvýši celková bezpečnosť a dôveryhodnosť celého systému.

2 SÚČASNÝ STAV RIEŠENEJ PROBLEMATIKY

V tejto kapitole podrobujem analýze aktuálne dostupné riešenia v oblasti digitálnej inzercie a zdieľanej ekonomiky. Zameriavam sa na to, ako svetoví lídri a lokálne slovenské portály prístupujú k správe služieb a prenájmu, pričom identifikujem funkčné a technologické nedostatky, ktoré moja platforma odstraňuje.

2.1 ANALÝZA KONKURENČNÝCH PLATFORIEM A SYSTÉMOV

Pre pochopenie štandardov v odvetví je nevyhnutné analyzovať systémy, ktoré definujú súčasné trendy v oblasti digitálnych trhovísk (marketplace). Tieto platformy reprezentujú rôzne logické prístupy k spracovaniu dát a budovaniu dôvery medzi používateľmi (Rochet a Tirole 2003). Na základe analýzy identifikujem ich silné stránky, ktoré som do vlastného riešenia prevzal, a zároveň ich obmedzenia, ktoré moja platforma prekonáva.

2.1.1 Airbnb.com (Model zdieľanej ekonomiky ubytovania)

Charakteristika a logika: Airbnb je prototypom modernej P2P platformy, ktorá zmenila celé odvetvie ubytovania. Platforma funguje ako dôveryhodný sprostredkovateľ medzi súkromnými prenajímateľmi a hosťami, pričom kľúčovú rolu hrá obojstranný systém hodnotení a overovanie identity (Sundararajan 2014).

Výhody: Sofistikovaný reputačný systém, geografické vyhľadávanie s mapovým rozhraním a dynamické ceny na základe dopytu a sezónnosti.

Nevýhody: Platforma je úzko špecializovaná na krátkodobý prenájom ubytovania. Vysoké poplatky pre hostiteľov (15–20 % z každej rezervácie) odrádzajú malých lokálnych prenajímateľov. Model tiež predpokladá štandardizovateľnosť ponuky, čo je pri remeslách a službách ťažko dosiahnuteľné.

Porovnanie s vlastným riešením: Z Airbnb preberám kľúčovú filozofiu: technológia musí prekonať nedôveru medzi cudzími ľuďmi. V mojom riešení je to realizované cez moderáciu inzerátov, systém nahlásení (Reports) a jasnú identitu každého inzerenta. Na rozdiel od Airbnb moja platforma neúčtuje provízie – monetizačný model je postavený na prémiových plánoch inzerentov (SellerPlan).

2.1.2 OLX / Bazoš.sk (Model všeobecnej inzercie)

Charakteristika a logika: OLX (pôsobí v strednej a východnej Európe) a jeho slovenský ekvivalent Bazoš.sk sú najrozšírenejšími platformami pre všeobecnú inzerciu v regióne. Modely sú postavené na maxime dosahu – veľký počet inzerátov, minimálne prekážky pre pridanie ponuky (Vallas a Schor 2020).

Výhody: Vysoká návštevnosť a povedomie značky. Jednoduché pridanie inzerátu bez registrácie. Pokrytie širokého spektra kategórií od elektroniky po nehnuteľnosti.

Nevýhody: Absencia moderácie vedie k množstvu neaktuálnych, duplicitných alebo podvodných inzerátov. Platforma nemá natívny chat ani systém správ – komunikácia prebieha cez zverejnené telefónne číslo, čo naruša súkromie a neumožňuje sledovanie konverzácií. Chýba geografická vizualizácia (mapa) a pokročilé filtrovanie. Vizuálne zastarané rozhranie necieli na mladšiu generáciu.

Porovnanie s vlastným riešením: Moja platforma rieši každý zo zmienených nedostatkov: povinná moderácia zabezpečuje kvalitu obsahu, integrovaný chat chráni súkromie používateľov a Leaflet mapa s GPS filtráciou umožňuje presné geografické vyhľadávanie.

2.1.3 Booking.com (Model komplexnej rezervácie a správy kapacít)

Charakteristika a logika: Booking.com operuje na báze pokročilých algoritmov pre správu voľných kapacít v reálnom čase (Rochet a Tirole 2003). Systém je postavený na okamžitej potvrdení rezervácii a dynamickej tvorbe cien, čo si vyžaduje robustnú architektúru na zamedzenie duplicitných rezervácií.

Výhody: Extrémne podrobné hierarchické filtrovanie a vysoká miera dôveryhodnosti budovaná cez overené recenzie.

Nevýhody: Systém je uzavretý výhradne pre segment ubytovania. Pre malých lokálnych poskytovateľov iných služieb sú prekážkou vysoké provízie a rigidita systému, ktorý neumožňuje inzerovať predmety dennej potreby alebo remeselné práce.

Porovnanie s vlastným riešením: Moja platforma preberá logiku podrobného filtrovania meta-údajov, avšak aplikuje ju univerzálne. Vďaka flexibilnej schéme v Prisma ORM umožňujem inzerátom priradiť rôzne špecifikácie (napr. technické parametre náradia) bez nutnosti meniť základnú architektúru databázy.

2.1.4 Fiverr.com (Model produktizácie služieb – Micro-gig economy)

Charakteristika a logika: Fiverr transformoval nehmotné služby na jasne definované balíčky (produkty) (Vallas a Schor 2020). Logika fungovania spočíva v eliminácii vyjednávania o cene a rozsahu práce, čím sa proces nákupu služby približuje klasickému elektronickému obchodu.

Výhody: Prehľadnosť nákupného procesu a automatizované filtrovanie kvality poskytovateľov cez systém úrovní.

Nevýhody: Orientácia takmer výhradne na digitálne produkty doručiteľné online. Absencia lokalizačnej vrstvy a máp znemožňuje efektívne sprostredkovanie fyzických služieb v konkrétnom meste.

Porovnanie s vlastným riešením: Implementujem logiku „služba ako produkt“, ale obohacujem ju o kľúčový geografický kontext. Integrácia mapových podkladov cez knižnicu Leaflet a ukladanie GPS súradníc v PostgreSQL umožňuje používateľom nájsť odborníka v ich bezprostrednom okolí.

2.1.5 Turo.com (Model Peer-to-Peer zdieľania majetku)

Charakteristika a logika: Turo reprezentuje model zdieľanej ekonomiky v doprave, kde súkromné osoby monetizujú svoj nevyužitý majetok (automobily) (Sundararajan 2014). Logika systému stojí na overovaní identity a synchronizácii s kalendárom vlastníka.

Výhody: Efektívne využitie existujúcich zdrojov a prepracovaná vizuálna prezentácia majetku.

Nevýhody: Ide o jednoúčelovú aplikáciu. Ak používateľ hľadá iný typ prenájmu (napr. stavebnú techniku), je nútený využiť ďalší samostatný systém.

Porovnanie s vlastným riešením: Navrhované riešenie predstavuje integrovaný ekosystém, ktorý konsoliduje fragmentované potreby lokálneho trhu do jedinej platformy, čím odstraňuje nutnosť využívania viacerých jednoúčelových aplikácií.

2.2 EKONOMICKÉ MODELY MODERNEJ INZERCIE

Pre pochopenie fungovania navrhovanej platformy je nevyhnutné definovať kľúčové ekonomické koncepty, ktoré v poslednom desaťročí transformovali globálny trh práce a služieb. Podľa správy Európskej komisie z roku 2021 sa obrat platforiem zdieľanej ekonomiky v EÚ od roku 2015 strojnásobil a v roku 2020 presiahol 60 miliárd eur, pričom segment služieb a prenájmu rastie najrýchlejšie (Vallas a Schor 2020).

2.2.1 Fenomén Micro-gig economy

Micro-gig economy (často označovaná aj ako „platformová ekonomika“) predstavuje model, v ktorom sú dlhodobé pracovné pomery nahrádzané krátkodobými, presne definovanými úlohami, takzvanými „gigmi“. Tento model zmenil vnímanie práce z fixného zamestnania na poskytovanie konkrétnej hodnoty v reálnom čase (Vallas a Schor 2020). Podľa odhadov Medzinárodnej organizácie práce (ILO) sa počet pracovníkov zapojených do platformovej ekonomiky celosvetovo medzi rokmi 2013 a 2020 zdvojnásobil.

Logika a fungovanie: Základom je digitalizácia dopytu. Platforma funguje ako algoritmus, ktorý páruje voľné kapacity dodávateľ a s okamžitou potrebou odberateľ a (Rochet a Tirole 2003). Na rozdiel od klasickej inzercie sa tu hľadá „riešenie problému“. Príkladom je platforma Fiverr, kde je služba predávaná ako vopred zabalený produkt s fixnou cenou a termínom dodania. Tento prístup eliminuje transakčné náklady spojené s vyjednávaním a zmluvnou dokumentáciou.

Výhody pre trh: Tento model umožňuje obrovskú mieru flexibility. Freelanceri môžu pracovať pre viacerých klientov súčasne a záujemcovia majú prístup k expertom bez nutnosti uzatvárania zložitých zmlúv. Pre lokálne trhy ako Slovensko je osobitne dôležitá možnosť, že aj malý remeselník dokáže osloviť zákazníkov v celom regióne bez investície do vlastného marketingu.

Riziká a výzvy: Gig economy čelí aj kritike – nedostatočná právna ochrana pra-

covníkov, nestabilný príjem a závislosť na algoritmickom hodnotení sú legitímne obavy (Vallas a Schor 2020). Moja platforma tieto riziká adresuje tým, že nepreberie rolu zamestnávateľa, ale zostáva neutrálnym sprostredkovateľom, ktorý dáva inzerentom plnú kontrolu nad podmienkami a cenou.

Aplikácia v mojom projekte: Moja platforma preberá z tohto modelu princíp „produk-tizácie” služieb. Vytvoril som systém, kde si používateľ nevyberá len osobu, ale konkrétnu ponuku s jasnými parametrami. Vďaka modulu `Advertisements` a dynamickým filtrom umožňujem inzerentom presne definovať rozsah svojej „mikro-zákazky” – vrátane ceny, lokality a technických špecifikácií – čím znižujem čas potrebný na vyjednávanie a zvyšujem transparentnosť transakcie.

2.2.2 Model Peer-to-Peer (P2P) a zdieľaná ekonomika

Model Peer-to-Peer (rovný s rovným) je základným stavebným kameňom zdieľanej ekonomiky. Ide o transakcie, ktoré prebiehajú priamo medzi dvoma koncovými používateľmi, pričom platforma vystupuje len ako technologický sprostredkovateľ a garant bezpečnosti (Sundararajan 2014).

Logika zdieľania majetku: Hlavnou myšlienkou je efektívna monetizácia nevyužitých statkov (Sundararajan 2014). V praxi to znamená, že osoba, ktorá vlastní napríklad rebríkový záves, elektrické náradie či pracovné stroje, môže tieto aktíva prenajímať susedovi namiesto toho, aby ležali nevyužitú. Tento prístup je nielen ekonomicky výhodný pre obe strany, ale aj ekologicky udržateľný – znižuje potrebu výroby nových výrobkov. Platformy ako Airbnb dokázali, že technológia dokáže prekonať prirodzenú nedôveru medzi cudzími ľuďmi a vybudovať fungujúce trhovisko na báze reputácie.

Budovanie dôvery – kľúčový problém P2P: Kľúčovým prvkom P2P logiky je reputačný systém a moderácia (Rochet a Tirole 2003). Bez možnosti nahlásiť nekorektné správanie, overiť totožnosť inzerenta alebo ohodnotiť transakciu by tento model skolaboval – potenciálnych záujemcov by odradila neistota. Štúdie ukazujú, že platformy s aktívnou moderáciou a systémom hodnotení dosahujú o 30–40 % vyššiu mieru opakovaného použitia (Sundararajan 2014).

Aplikácia v mojom projekte: Moja platforma je postavená na P2P princípe najmä v segmente prenájmu. Implementoval som moduly `Messages` pre bezpečnú komunikáciu (používateľ nikdy nemusí zverejniť svoje telefónne číslo), `Reports` pre nahlásovanie incidentov a systém moderácie, ktorý zaručuje, že na platforme sa objavia len overené inzeráty. Na rozdiel od platforiem ako Bazoš môj systém vďaka administrátorskému panelu umožňuje aktívne zasahovať, sledovať trendové kategórie a chrániť používateľov v reálnom čase.

2.2.3 Slovenský trh a lokálna príležitosť

Slovenský trh digitálnych služieb a prenájmu vykazuje špecifické charakteristiky, ktoré vytvárajú priestor pre lokálne riešenie. Dominantné platformy ako Bazoš.sk slúžia primárne predaju tovaru, nie sprostredkovaniu služieb a prenájmu. Portál pretlak.com pokrýva len kreatívne a IT profesie. Výsledkom je, že desiatky tisíc remeselníkov, živnostníkov a malých poskytovateľov služieb nemajú k dispozícii modernú digitálnu vitrínu, ktorá by zodpovedala ich potrebám (Vallas a Schor 2020).

Tento „biely priestor“ na trhu predstavuje hlavnú podnikateľskú príležitosť navrhovanej platformy RentMe. Cieľom nie je konkurovať globálnym gigantom, ale obsadiť lokálny segment, kde veľkí hráči nie sú prítomní alebo kde ich riešenia nie sú prispôbené lokálnym zvyklostiam, jazyku a právnemu prostrediu.

2.3 TECHNOLOGICKÉ TRENDY A PRIDANÁ HODNOTA RIEŠENIA

Aby aplikácia obstála v konkurencii, využíva moderný technologický stack, ktorý rieši slabiny starších portálov. Volba technológií nie je len technické rozhodnutie – priamo ovplyvňuje rýchlosť vývoja, bezpečnosť systému a dlhodobú udržateľnosť kódu (Winters et al. 2020).

Server-side Rendering (Next.js): Zabezpečuje rýchle načítanie a lepšiu indexáciu inzerátov vyhľadávačmi (SEO), čo je kľúčové pre návštevnosť verejnej platformy (Riva 2022).

Typová bezpečnosť (TypeScript): Použitie monorepa umožňuje zdieľať dátové modely (DTO) medzi API a frontend aplikáciami, čím sa minimalizuje riziko nekonzistencií v dátach (Cherny 2019).

Analytika (ClickEvent): Na rozdiel od neformálnych riešení (Facebook skupiny) systém monitoruje správanie používateľov (kliknutia, segmentácia podľa pohlavia či typu účtu), čo umožňuje administrátorovi optimalizovať kategórie podľa skutočného dopytu (Winters et al. 2020).

3 ANALÝZA A NÁVRH SYSTÉMU

Návrh systému vychádza z rozdelenia platformy na štyri základné časti: verejná platforma, používateľské konto, admin panel a backend API (Sommerville 2015). Tento prístup, realizovaný pomocou monorepo štruktúry a oddelených `Next.js` aplikácií, umožňuje presne definovať oprávnenia pre rôzne skupiny používateľov a izolovať ich používateľské rozhrania.

3.1 FUNKČNÉ POŽIADAVKY

Funkčné požiadavky sú rozdelené podľa rolí, ktoré v systéme vystupujú (Sommerville 2015). Každá rola má priradený špecifický rozsah akcií, ktoré môže v rámci aplikácie vykonávať. Na úrovni databázy je rola používateľ a definovaná enumerátorom `UserRole` (s hodnotami `ADMIN` a `USER`).

Obrázok 1 zobrazuje diagram použitia (Use Case): vzťahy medzi aktérmi (Návštevník, Používateľ, Administrátor) a vybranými prípadmi použitia. Pri aktérovi Používateľ je šípkou zovšeobecnenia (UML generalization) znázornené, že okrem vlastných prípadov použitia zdedí aj možnosti návštevníka.

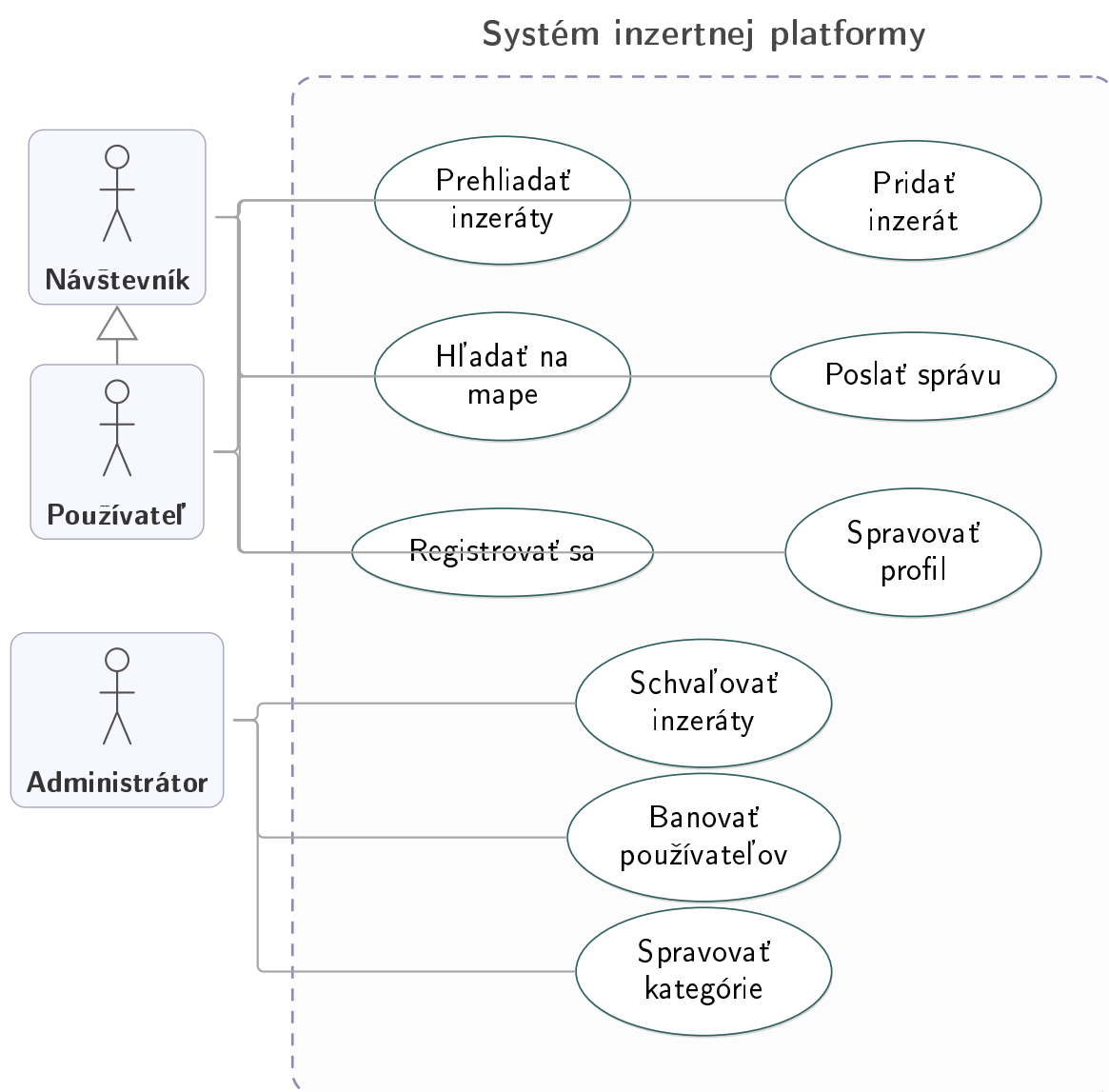
3.1.1 Rola: Návštevník (neprihlásený používateľ)

Návštevník je anonymný používateľ, ktorý primárne konzumuje verejne dostupný obsah bez potreby registrácie.

- **Prehliadanie obsahu:** Používateľ môže prehliadať kategórie, zoznamy inzerátov a detaily jednotlivých ponúk.
- **Vyhľadávanie a filtrácia:** Možnosť vyhľadávať inzeráty podľa kľúčových slov a filtrovať výsledky na základe dynamických parametrov kategórie.
- **Mapové zobrazenie:** Prístup k interaktívnej mape, ktorá vizualizuje polohu inzerovaných služieb a prenájmov.
- **Čítanie blogu:** Možnosť čítať publikované články a statické informačné stránky (napr. podmienky používania).
- **Registrácia:** Návštevník má možnosť vytvoriť si používateľský účet.

3.1.2 Rola: Používateľ (registrovaný člen)

Používateľ má po prihlásení prístup k funkciám, ktoré umožňujú aktívnu participáciu na trhu služieb. Prístup k týmto funkciám je na backende chránený pomocou JWT autentifikácie (v `NestJS` implementovanej cez `JwtAuthGuard`).



Obr. 1: Diagram použitia (Use Case) – funkčné požiadavky a aktéri systému

- **Správa profilu:** Úprava osobných údajov, firemných údajov (napr. IČO, DIČ) a zmena prístupového hesla.
- **Manažment inzerátov:** Tvorba nových inzerátov (služba alebo prenájom), ich úprava, mazanie a sledovanie stavu schválenia. V databáze môže inzerát nadobúdať stavy definované enumerátorom `AdvertisementStatus` (`PENDING`, `DRAFT`, `ACTIVE`, `INACTIVE`, `ARCHIVED`).
- **Komunikačný modul:** Odosielanie a prijímanie správ (dopytov) k inzerátom prostredníctvom integrovaného chatu.
- **Obľúbené položky:** Ukladanie zaujímavých inzerátov do vlastného zoznamu pre neskorší návrat (entita `Favorite`).
- **Nahlasovanie (Reportovanie):** Možnosť nahlásiť inzerát, ktorý porušuje pravidlá platformy.

3.1.3 Rola: Administrátor

Administrátor má najvyššiu úroveň oprávnení a dohliada na technický a obsahový chod platformy cez dedikovaný admin panel. Na úrovni API sú tieto endpointy chránené kombináciou `JwtAuthGuard` a vlastného `RolesGuard`, ktorý overuje, či má prihlásený používateľ rolu `ADMIN`.

```
@Controller('admin')
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(UserRole.ADMIN)
export class AdminController {
  // ...
  @Get('stats')
  getStats() {
    return this.adminService.getStats();
  }
}
```

Výpis 1: Ukážka zabezpečenia administrátorského endpointu v NestJS.

- **Moderácia obsahu:** Schvaľovanie, zamietanie alebo archivácia inzerátov a riešenie prijatých nahlásení od používateľov.
- **Správa používateľov:** Prehľad o registrovaných členoch s možnosťou udelenia dočasného alebo trvalého banu (blokovania účtu). Databázový model `User` obsahuje na tento účel polia `banned`, `bannedUntil` a `banReason`.

- **CMS (Content Management System):** Tvorba a editácia statických stránok, blogových príspevkov a správa položiek v navigačnom menu.
- **Konfigurácia systému:** Nastavovanie kategórií, hierarchie filtrov a globálnych parametrov platformy.
- **Analytika a prehľady:** Sledovanie štatistík návštevnosti, kliknutí (`ClickEvent`) a trendov v inzercii prostredníctvom grafov.

3.2 NEFUNKČNÉ POŽIADAVKY

Nefunkčné požiadavky definujú technické limity a prevádzkové vlastnosti, ktoré sú nevyhnutné pre spoľahlivý chod platformy (Sommerville 2015). Pri systéme, ktorý sprostredkovať interakciu medzi cudzími ľuďmi a narába s ich citlivými údajmi, som kládol dôraz predovšetkým na bezpečnosť, jednoduchú rozšíriteľnosť a rýchlu odozvu používateľského rozhrania.

3.2.1 Bezpečnosť a ochrana údajov

Vzhľadom na to, že platforma spracováva používateľské profily a súkromnú komunikáciu, je bezpečnosť prioritou.

Autentifikácia a autorizácia: Prístup k citlivým častiam API je chránený platným JSON Web Tokenom (JWT). V prostredí `NestJS` je táto vrstva realizovaná pomocou stratégie `passport-jwt` a chránená globálnymi guardmi (`JwtAuthGuard`, `RolesGuard`), ktoré pri každej požiadavke overujú identitu aj priradenú rolu používateľa.

Hashovanie hesiel: Heslá nie sú v databáze `PostgreSQL` nikdy ukladané v čitateľnej podobe. Na ich bezpečnú transformáciu využívam kryptografický algoritmus `bcrypt` so soľou (salt), čo znemožňuje ich spätné dešifrovanie.

Validácia vstupov: Aby som predišiel útokom typu *mass assignment* alebo spracovaniu neúplných dát, systém automaticky filtruje každú prichádzajúcu požiadavku pomocou globálneho `ValidationPipe`. Tento nástroj pracuje v spolupráci s knižnicou `class-validator` a DTO triedami – polia, ktoré nie sú v DTO definované, sú automaticky odmietnuté. Toto opatrenie bráni scenáru, pri ktorom by útočník poslal nepovolené polia (napr. `role: "ADMIN"`) s cieľom zmeniť citlivé atribúty objektu.

CORS a blokovanie zneužitia: API má presne definované povolené zdroje (origins), čím sa bráni neautorizovaným volaniam z cudzích domén. Zároveň systém obsahuje mechanizmus banovania, ktorý administrátorovi umožňuje okamžite zablokovat' prístup nahláseným používateľom (využitím polí `banned` a `banReason` v databáze).

3.2.2 Škálovateľnosť a udržateľnosť systému

Platforma je navrhnutá tak, aby bolo možné v budúcnosti jednoducho pridávať nové funkcie bez nutnosti prepisovania existujúceho kódu.

Architektúra monorepa: Využitie `npm workspaces` mi umožnilo logicky oddeliť frontendové aplikácie (`platform`, `user`, `admin`) od backendu (`api`). Vďaka zdieľanému balíku `packages/shared` sú dátové typy a DTO konzistentné naprieč celým projektom, čo výrazne znižuje chybovosť pri vývoji.

Modularita a ORM: Backend je rozdelený do samostatných doménových modulov (napr. `AuthModule`, `Advertisements Module`), čo uľahčuje údržbu a testovanie. Použitie `Prisma ORM` zasa zabezpečuje, že zmeny v dátovom modeli sú typovo bezpečné a ľahko nasaditeľné pomocou verziovaných migrácií.

Kontajnerizácia: Celý projekt je pripravený na nasadenie cez Docker. Súbor `docker-compose.yml` definuje izolované prostredia pre databázu aj Node.js služby, čo zaručuje, že aplikácia sa bude správať identicky na mojom počítači aj na produkčnom serveri.

3.2.3 Výkon, odozva a SEO

Rýchlosť načítania a viditeľnosť vo vyhľadávačoch sú kľúčové pre úspech každého inzertného portálu.

Server-side Rendering (SSR): Vďaka `Next.js` (verzia 16+) sa obsah inzerátov generuje priamo na serveri. To zabezpečuje nielen okamžitú odozvu pre používateľ'a, ale aj ideálnu indexáciu Google robotmi, keďže HTML dokument už pri načítaní obsahuje dôležité SEO metadáta.

Optimalizácia databázy: Pre zrýchlenie vyhľadávania pri veľkom množstve dát som v Prisma schéme definoval indexy (`@@index`) pre najčastejšie filtrované polia, ako sú `status`, `slug` či `categoryId`.

Responzívny dizajn: Používateľské rozhranie postavené na `Tailwind CSS` využíva prístup `mobile-first`. To zaručuje, že platforma je plne funkčná a vizuálne prítiahľivá na smartfónoch aj veľkých monitoroch.

Práca s médiami: API je optimalizované na spracovanie väčších objemov dát (limit 50 MB), čo umožňuje bezproblémový prenos obrázkov inzerátov kódovaných vo formáte `base64`.

3.3 NÁVRH ARCHITEKTÚRY: MODEL KLIENT–SERVER V PROSTREDÍ MONOREPA

Architektúra systému je postavená na modernom koncepte monorepa, ktorý umožňuje efektívnu správu viacerých samostatných aplikácií a zdieľaných knižníc v rámci jedného úložiska

(repozitára) (Winters et al. 2020). Celý ekosystém je rozdelený na klientske aplikácie (front-end) a centrálny server (back-end), ktoré medzi sebou komunikujú prostredníctvom protokolu HTTP a formátu JSON.

3.3.1 Štruktúra monorepa a zdieľanie kódu

Projekt využíva nástroj `npm workspaces`, ktorý zabezpečuje logické rozdelenie kódu do dvoch hlavných kategórií: aplikácie (`apps/`) a balíčky (`packages/`).

- **Aplikácie (`apps/`):** Obsahujú tri nezávislé `Next.js` aplikácie s App Routerom (verzia 16+): verejnú platformu (`apps/platform`), používateľské konto (`apps/user`) a administráciu (`apps/admin`). Štvrtou aplikáciou je centrálny `NestJS` API server (`apps/api`).
- **Zdieľané balíčky (`packages/`):** Kľúčovým prvkom je balík `packages/shared`, ktorý obsahuje spoločné TypeScript typy, enumerátory (napríklad `UserRole` a `AdvertisementStatus`) a objekty DTO (Data Transfer Objects). Vďaka tomu je zaručené, že ak sa zmení štruktúra dát na backende, zmena sa okamžite premietne aj do frontendu, čím sa predchádza nekonzistenciám a chybám pri kompilácii.
- **Databázový balík:** Balík `packages/database` obsahuje definíciu Prisma schémy (`schema.prisma`), vygenerovaného Prisma klienta a migračné skripty. Týmto je prístup k databáze centralizovaný a zdieľaný pre potreby API.

3.3.2 Model klient–server

Systém funguje na princípe striktného oddelenia prezentácie od aplikačnej logiky a ukladania dát.

- **Backend API (Server):** Je postavený na frameworku `NestJS` a funguje ako centrálny bod systému. Zabezpečuje komunikáciu s databázou `PostgreSQL` cez `Prisma ORM`, vykonáva striktnú validáciu dát a overuje oprávnenia používateľov cez JWT tokeny (pomocou `Passport.js`). API zároveň poskytuje automaticky generovanú Swagger dokumentáciu na adrese `/api/docs`.
- **Klientske aplikácie (Klienti):** Tri samostatné aplikácie v `Next.js` komunikujú s API prostredníctvom asynchrónnych HTTP požiadaviek. URL backendu je definované premennou prostredia `NEXT_PUBLIC_API_URL`. Každá aplikácia má vlastné, optimalizované rozhranie – vizuálny editor `GrapesJS` v admin paneli, interaktívna mapa `Leaflet` na verejnej platforme.

3.3.3 Dátový tok a komunikácia

Keď používateľ vykoná akciu (napríklad pridá nový inzerát), proces prebieha v nasledujúcich krokoch:

1. **Iniciácia na klientovi:** Frontend (napr. `apps/user`) zhromažďí dáta z formulára a odošle HTTP `POST` požiadavku. Dáta sú už na strane klienta typované podľa zdieľaných DTO typov z balíka `packages/shared`. Do hlavičky `Authorization` je vložený JWT token prihláseného používateľa.
2. **Prijatie a validácia na serveri:** API server prijme požiadavku. Najprv overí identitu používateľa cez `JwtAuth Guard` (prípadne rolu cez `Roles Guard`). Následne globálna `Validation Pipe` skontroluje telo požiadavky voči DTO triede (odstráni neznáme polia a overí povinné parametre).
3. **Zápis do databázy:** Ak sú dáta validné, príslušný kontrolér posunie požiadavku do aplikačnej služby. Tá pomocou `Prisma ORM` vykoná asynchrónny zápis do databázy `PostgreSQL`.
4. **Odpoveď servera:** Server vráti odpoveď v JSON formáte (napr. vytvorený objekt inzerátu s vygenerovaným UUID a HTTP status kódom `201 Created`).
5. **Aktualizácia používateľského rozhrania:** Frontend prijme odpoveď a zareaguje aktualizáciou rozhrania (napr. presmerovaním používateľa na detail novovytvoreného inzerátu alebo zobrazením notifikácie o úspechu).

Obrázok 2 znázorňuje zjednodušený tok vrstiev pri zápise po úspešnej validácii (krok 3), v súlade s architektúrou NestJS a perzistenciou cez Prisma.

3.4 DATABÁZOVÝ MODEL A ENTITNO-RELAČNÝ DIAGRAM (ERD)

Pre potreby perzistencie dát využívam relačnú databázu `PostgreSQL`, ktorá je v rámci vývoja spravovaná pomocou nástroja `Prisma ORM` (Obe a Hsu 2017). Tento prístup mi umožňuje definovať dátový model v prehľadnom deklaratívnom formáte (`schema.prisma`) a následne automaticky generovať migračné skripty a typovo bezpečný klient pre backendovú aplikáciu.

3.4.1 Hlavné entity systému a ich význam

Logický model databázy pozostáva z niekoľkých kľúčových entít, ktoré sú navzájom prepojené reláciami typu 1:N (one-to-many) alebo M:N (many-to-many).



Obr. 2: Tok zápisu dát po validácii: kontrolér, aplikačná služba, Prisma ORM a databáza PostgreSQL

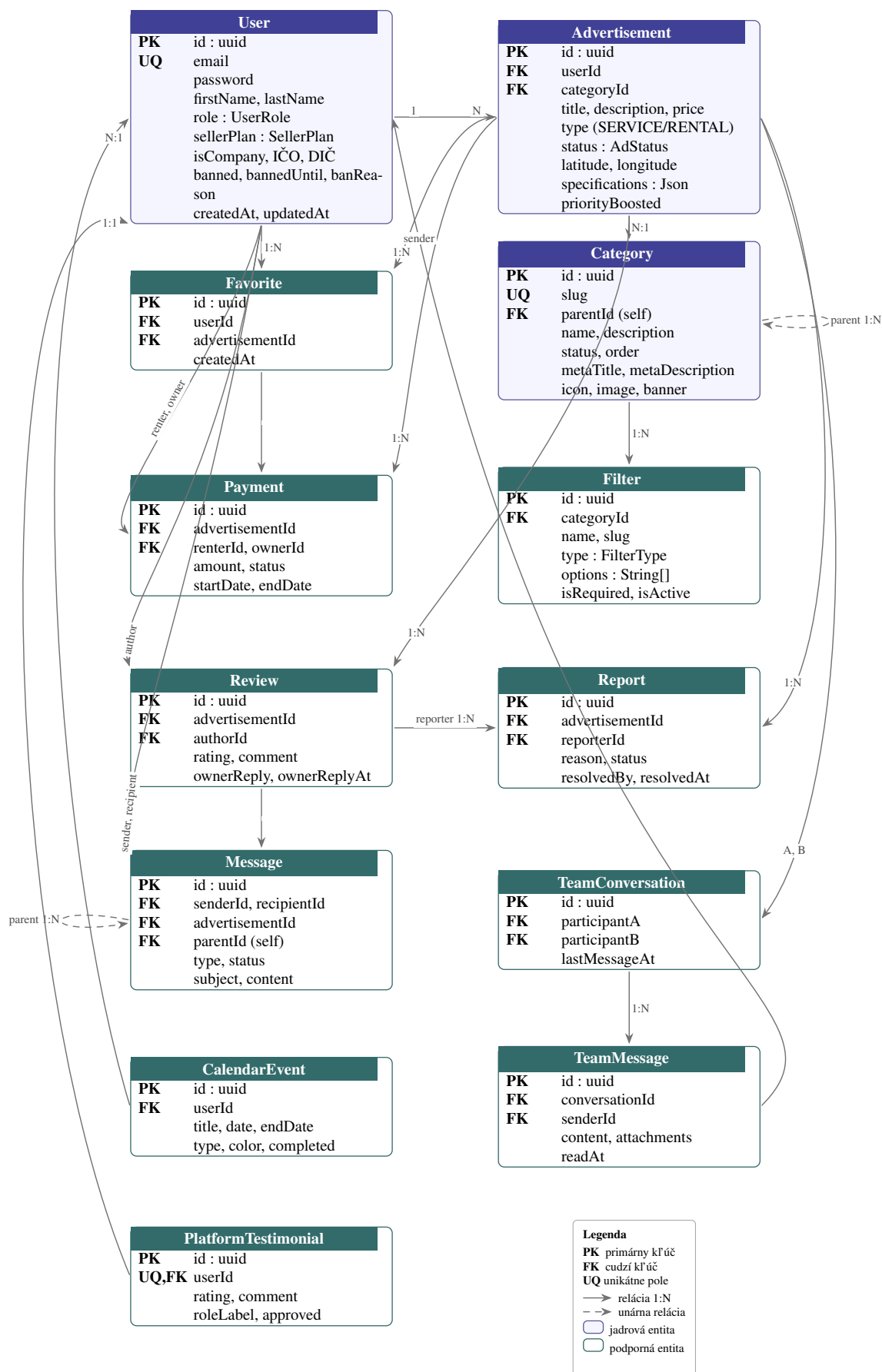
- **User (Používateľ):** Predstavuje identitu v systéme. Okrem základných údajov ako e-mail a hashované heslo obsahuje informácie o roli (`ADMIN` / `USER`), firemné údaje pre predajcov (`IČO`, `DIČ`, `isCompany`), predajný plán (`SellerPlan`) a polia súvisiace s moderáciou, ako sú `banned`, `bannedUntil` a `banReason`.
- **Advertisement (Inzerát):** Centrálna entita obsahujúca detaily o ponúkanej službe alebo prenájme (`AdvertisementType`). Inzerát je viazaný na konkrétneho autora (používateľa) a kategóriu. Obsahuje stavy definované enumerátorom `AdvertisementStatus` (`PENDING`, `DRAFT`, `ACTIVE`, `INACTIVE`, `ARCHIVED`), lokalitu (GPS súradnice `latitude`, `longitude`), cenu a špecifikácie uložené vo formáte JSON (`specifications`) pre maximálnu flexibilitu parametrov bez nutnosti meniť databázovú schému pri každom novom parametri.
- **Category (Kategória):** Zabezpečuje hierarchické usporiadanie inzerátov. Podporuje vnorenie (parent-child relácie cez `parentId`), obsahuje SEO polia (`metaTitle`, `metaDescription`, `slug`) a definuje poradie zobrazenia v navigácii (`order`).
- **Filter (Dynamický filter):** Úzko spolupracuje s kategóriami. Definuje, aké dodatočné parametre (napr. typ `TEXT`, `NUMBER`, `SELECT`, `MULTISELECT`, definované v `FilterType`) môže používateľ pri inzeráte v danej kategórii zadať a podľa čoho môže návštevník vyhľadávať. Umožňuje tak dynamické rozširovanie vlastností inzerátov podľa ich zaradenia.

- **Message (Správa):** Realizuje komunikačný kanál medzi záujemcom a inzerentom, ale aj systémové notifikácie (`MessageType`). Správy sú logicky zoskupované do konverzácií (vlákién) pomocou odkazu na rodičovskú správu (`parentId`), čo umožňuje prehľadné zobrazenie chatu v používateľskom rozhraní.
- **Favorite (Obľúbené):** Spojovacia entita realizujúca reláciu M:N medzi používateľom a inzerátmi, ktoré si uložil na neskoršie prehlíadnutie.
- **Report (Nahlásenie):** Slúži na moderáciu obsahu. Uchováva informáciu o sťažnosti na konkrétny inzerát, dôvod nahlásenia (`ReportReason` – napr. `SPAM`, `SCAM`, `FAKE`) a stav riešenia administrátorom (`ReportStatus`).
- **Payment (Platba):** Zaznamenáva finančné transakcie medzi záujemcom (`renter`) a vlastníkom inzerátu (`owner`), pričom sleduje stav platby (`PaymentStatus`) a obdobie prenájmu/poskytnutia služby.
- **StaticPage a BlogPost (CMS obsah):** Slúžia na správu obsahu priamo z admin panelu. Obsahujú HTML text, SEO metadáta a stav publikovania (`DRAFT` / `PUBLISHED`), čím umožňujú administrátorom upravovať informačné podstránky a blog bez zásahu do kódu.
- **ClickEvent (Analytika):** Špeciálna entita určená na zber údajov o správaní používateľov. Zaznamenáva typ interakcie (`eventType`), cieľový inzerát alebo kategóriu a segmentačné údaje (pohlavie, typ účtu), čo slúži ako podklad pre administratívne štatistiky a grafy.

3.4.2 Entitno-relačný diagram (ERD)

Logický model databázy som vytvoril v deklaratívnom jazyku Prisma (`schema.prisma`), z ktorého následne vyplýva fyzická schéma v PostgreSQL. Obrázok 3 zobrazuje úplný entitno-relačný diagram so všetkými hlavnými tabuľkami, kľúčmi (PK, FK, UQ) a kardinalitou jednotlivých relácií. Jadrové entity (`User`, `Advertisement`, `Category`) sú vizuálne odlíšené modrastou farbou, podporné a doménové tabuľky sú zobrazené v zelenom (teal) prevedení. Vybrané charakteristické relácie:

- Relácia medzi `User` a `Advertisement` je typu **1:N** – jeden používateľ môže vytvoriť viacero inzerátov, no inzerát patrí vždy jednému autorovi.
- `Category` a `Advertisement` tvoria vzťah **1:N**, kde každá kategória zastrešuje množinu inzerátov.



Obr. 3: Entitno-relačný diagram (ERD) databázy – jadrové entity, podporné tabuľky a ich relácie

- `Category` obsahuje aj **unárnu reláciu 1:N** sama na seba (cez `parentId`), čím vzniká stromová štruktúra hlavných a podriadených kategórií.
- `Filter` je viazaný na konkrétnu kategóriu (1:N) a definuje dynamické vlastnosti inzerátov v tejto kategórii.
- `Favorite` predstavuje klasickú dekompozíciu vzťahu **M:N** medzi používateľom a inzerátom.
- `Payment` prepája inzerát s dvoma používateľmi (`renter` a `owner`), čo vytvára komplexnejšiu sieť vzťahov nad tabuľkou používateľov.
- `Message` obsahuje vlastnú **unárnu reláciu** (`parentId` → `replies`), ktorá umožňuje hierarchické zoradovanie odpovedí do vlákien konverzácií.
- `TeamConversation` viaže vždy dvoch používateľov cez polia `participantA` a `participantB` (dve nezávislé 1:N relácie), pričom unikátnym obmedzením je zaručené, že medzi dvoma účastníkmi existuje vždy najviac jedna konverzácia.
- `PlatformTestimonial` má voči `User` reláciu **1:1** – jeden používateľ môže mať najviac jednu verejnú referenciu zobrazenú na homepage.

Anotácie `@@index` v Prisma schéme explicitne deklarujú indexy nad poliami, ktoré sa najčastejšie objavujú v podmienkach `WHERE` (`status`, `slug`, `categoryId`, `userId`, `createdAt`), čím sa zaručuje predvídateľný výkon dopytov aj pri raste objemu dát. Klauzuly `onDelete: Cascade` a `onDelete: SetNull` explicitne riadia referenčnú integritu na úrovni databázy – odstránenie agregátneho koreňa (napr. používateľ a) automaticky odstráni alebo zneplatní viazané záznamy bez nutnosti špeciálnej aplikačnej logiky.

3.5 NÁVRH REST API ROZHRAŇA A AUTENTIFIKAČNÉHO MECHANIZMU (JWT)

Komunikácia v systéme prebieha prostredníctvom REST API (Representational State Transfer), ktoré využíva protokol HTTP a formát JSON na výmenu údajov. API je navrhnuté ako bezstavové, čo znamená, že každá požiadavka od klienta musí obsahovať všetky informácie potrebné na jej spracovanie, vrátane overenia identity (Richardson et al. 2013).

3.5.1 Návrh endpointov a smerovania

API je logicky rozdelené do modulov, ktoré zodpovedajú doménam v databáze. Každý modul definuje sadu koncových bodov (endpointov) pre operácie CRUD (Create, Read, Update, Delete) a špecifickú biznis logiku.

Modul	Základná cesta	Významné operácie
Auth	/auth	Registrácia, prihlásenie, získanie profilu prihláseného používateľa.
Advertisements	/advertisements	Vyhľadávanie, tvorba inzerátov, schvaľovanie (admin), mapa.
Messages	/messages	Odosielanie dopytov, správa konverzácií a chat.
Analytics	/analytics	Zaznamenávanie kliknutí (/click) a generovanie reportov.

Tabuľka 1: Prehľad hlavných modulov API a základných ciest

Všetky endpointy sú zdokumentované pomocou nástroja Swagger, ktorý je prístupný na adrese `GET /api/docs`. Táto dokumentácia slúži ako kontrakt medzi backendom a frontendovými tímami (v tomto prípade aplikáciami `platform`, `user` a `admin`).

3.5.2 Autentifikačný mechanizmus (JWT)

Na zabezpečenie prístupu k chráneným zdrojom využíva platforma JSON Web Token (JWT) (Jones et al. 2015). Tento mechanizmus umožňuje overiť identitu používateľa bez nutnosti ukladať informácie o relácii (session) na serveri.

Proces autentifikácie a autorizácie:

1. **Prihlásenie:** Používateľ odošle prihlasovacie údaje na `/auth/login`. Server overí heslo (pomocou `bcrypt` s 10 salt rounds) a ak je správne, vygeneruje podpísaný JWT token.
2. **Payload tokenu:** Token obsahuje zakódované informácie ako `userId`, `email` a `role` (`ADMIN` alebo `USER`).
3. **Autorizované požiadavky:** Klient ukladá token a posiela ho v hlavičke každého ďalšieho volania: `Authorization: Bearer <token>`.
4. **Validácia na serveri:** API využíva `JwtAuth Guard` na overenie podpisu tokenu a `Roles Guard` na kontrolu, či má používateľ dostatočné oprávnenia (napr. prístup do admin panelu).

3.5.3 Validácia a transformácia údajov

Pre zabezpečenie integrity systému je každá prichádzajúca požiadavka validovaná pomocou DTO (Data Transfer Objects). Využitie globálneho `ValidationPipe` v `NestJS` zabezpečuje:

- **Whitelist:** Odstránenie polí, ktoré nie sú definované v DTO, čím sa bráni neoprávneným zmenám v databáze.
- **Transformácia:** Automatický prevod reťazcov z URL parametrov na číselné typy alebo objekty, čo zjednodušuje prácu v službách (services).

3.5.4 Spracovanie chýb (Error Handling)

API využíva štandardizované HTTP status kódy (napr. 400 Bad Request pre chyby validácie, 401 Unauthorized pre neautorizovaný prístup, 403 Forbidden pre nedostatočné práva a 404 Not Found pre nenájdene zdroje). Systém vracia jednotný formát chybových hlásení vo formáte JSON, čo umožňuje ich ľahšie a konzistentné spracovanie na strane klientskych aplikácií (frontendu).

Tento návrh API a bezpečnosti zaručuje, že platforma je robustná, odolná voči bežným útokom a pripravená na komunikáciu s rôznymi typmi klientskych zariadení.

4 METODIKA A POUŽITÉ TECHNOLOGIE

Pri vývoji komplexnej inzertnej platformy som zvolil prístup, ktorý kladie dôraz na rýchlosť vývoja, bezpečnosť kódu a jednoduchú údržbu. Celý ekosystém je postavený na moderných JavaScriptových technológiách, ktoré umožňujú zdieľanie logiky medzi serverovou a klientskou časťou.

4.1 FULL-STACK VÝVOJ V JAZYKU TYPESCRIPT

Základným stavebným kameňom celého projektu je jazyk *TypeScript* (Cherny 2019). Ide o nadstavbu JavaScriptu, ktorá do dynamického sveta webového vývoja prináša statické typovanie. Pre projekt tohto rozsahu, kde komunikujú tri frontendové aplikácie s jedným API, je použitie TypeScriptu kľúčovým rozhodnutím z niekoľkých dôvodov:

4.1.1 Typová bezpečnosť a eliminácia chýb

TypeScript umožňuje definovať presné štruktúry dát (rozhrania a typy) už počas písania kódu. Vďaka tomu dokáže vývojové prostredie (IDE) okamžite upozorniť na chyby, ktoré by sa v čistom JavaScripte prejavili až počas behu aplikácie (napr. prístup k neexistujúcemu polu objektu). V projekte sú takto ošetrené všetky kľúčové entity, ako sú inzeráty, používatelia či správy. Zatiaľ čo TypeScript zabezpečuje internú konzistenciu kódu počas vývoja (*compile-time*), v kombinácii s knižnicou `class-validator` na backende garantuje, že do systému nevstúpia nevalidné dáta ani z externých HTTP požiadaviek (*runtime*).

4.1.2 Zdieľanie kódu v monorepe

Využitie TypeScriptu v prostredí monorepa mi dovoľuje vytvoriť balík `packages/shared`, kde sú definované DTO (*Data Transfer Objects*). Tieto objekty slúžia ako záväzný kontrakt medzi backendom a frontendom.

Keď v `api` zmením štruktúru odpovede pre detail inzerátu, TypeScript ma v aplikáciách `platform` alebo `admin` automaticky upozorní na všetkých miestach, kde je potrebné kód aktualizovať.

Tento mechanizmus zabezpečuje *end-to-end* typovú bezpečnosť, čo dramaticky zvyšuje stabilitu systému pri zmenách v databázovom modeli.

Príklad zdieľaného DTO v balíku `packages/shared` (využitie tried pre runtime validáciu):

```
import {
  IsString, IsOptional, IsNumber,
} from 'class-validator';

export class CreateAdvertisementDto {
  @IsString()
  title: string;

  @IsString()
  description: string;

  @IsOptional()
  @IsNumber()
  price?: number;

  // ...
}
```

Poznámka: V NestJS sa pre DTO odporúča používať triedy (class) namiesto rozhraní (interface).

Dôvodom je, že rozhrania v JavaScripte po kompilácii úplne zmiznú, kým triedy zostávajú zachované aj v runtime.

To umožňuje knižniciam ako `class-validator` alebo Swagger čítať metadáta o typoch polí.

4.1.3 Lepšia čitateľnosť, dokumentácia a Developer Experience (DX)

Kód napísaný v TypeScript je v podstate „samodokumentovateľný“. Akýkoľvek iný vývojár (alebo administrátor systému) dokáže z definícií typov okamžite pochopiť, aké dáta funkcia prijíma a čo vracajú jednotlivé API endpointy. To je dôležité najmä pri modulárnej architektúre NestJS, kde sú jednotlivé služby a kontroléry jasne definované pomocou dekorátorov a typov.

Zároveň TypeScript výrazne zlepšuje *Developer Experience (DX)* vďaka autokompletizácii (IntelliSense) v moderných IDE (ako je VS Code). Pre vývojára to znamená, že nemusí neustále prepínať okná a pozeráť sa do databázovej schémy, pretože mu editor sám našepkáva dostupné polia inzerátu alebo používateľa.

4.1.4 Moderné funkcie, dekorátory a kompatibilita

TypeScript mi umožňuje využívať najnovšie funkcie jazyka ECMAScript (ako asynchrónne funkcie, deštrukturalizáciu či voliteľné reťazenie), ktoré sú následne kompilované do optimalizovaného JavaScriptu. To zaručuje, že aplikácia beží efektívne v prostredí Node.js na serveri aj v moderných webových prehliadačoch.

Keďže backend využíva NestJS, naplno sa tu prejavuje sila TypeScriptu aj cez dekorátory (`@Controller`, `@Get`, `@Injectable`). Tie umožňujú reflexiu metadát, čo je pokročilá funkcia, vďaka ktorej NestJS vie, ako poskladať závislosti (*Dependency Injection*).

4.2 BACKEND: FRAMEWORK NESTJS A MODULÁRNA ARCHITEKTÚRA

Backendová časť aplikácie (`apps/api`) je vyvinutá v prostredí NestJS. Ide o progresívny framework pre Node.js (Casciaro a Mammino 2020), ktorý využíva TypeScript a kombinuje prvky objektovo orientovaného programovania, funkcionálneho programovania a reaktívneho programovania.

4.2.1 Modulárna štruktúra a organizácia kódu

Hlavnou výhodou NestJS je jeho striktná modulárna architektúra (Martin 2017). Každá logická časť platformy je zapuzdrená do samostatného modulu, ktorý obsahuje vlastné kontroléry, služby a dátové modely.

- **Zapuzdrenie logiky:** Napríklad `AdvertisementsModule` sa stará výhradne o inzeráty, zatiaľ čo `AuthModule` rieši autentifikáciu. Tieto moduly sú navzájom nezávislé a striktno dodržiavajú princíp jednej zodpovednosti (*Single Responsibility Principle* – SRP), čo uľahčuje testovanie a umožňuje pridávanie nových funkcií (napr. platobný modul) bez rizika narušenia existujúceho kódu. Všetky tieto doménové moduly sú následne spojené do jedného hlavného stromu aplikácie prostredníctvom koreňového modulu (`app.module.ts`).
- **Dekorátory a metadáta:** NestJS intenzívne využíva dekorátory (napr. `@Get()` , `@Post()` , `@UseGuards()`), ktoré robia kód čitateľnejším a deklaratívnym. Už na prvý pohľad je jasné, ktorý endpoint je verejný a ktorý vyžaduje administrátorské oprávnenia.

Príklad použitia dekorátorov v kontroléri (ukážka z `AdvertisementsController`):

```
@Controller('advertisements')
export class AdvertisementsController {
  constructor(private readonly advertisementsService:
    AdvertisementsService) {}

  @Get('pending/all')
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(UserRole.ADMIN)
  findPending() {
    return this.advertisementsService.findPending();
  }
}
```

V tejto ukážke je jasne vidieť, ako dekorátory `@UseGuards` a `@Roles` zabezpečujú ochranu endpointu tak, aby k nemu mal prístup iba administrátor.

4.2.2 Dependency Injection (Injektovanie závislostí)

Framework využíva návrhový vzor *Dependency Injection* (DI), ktorý automatizuje správu inštancií tried (Martin 2017).

V praxi to znamená, že ak služba `AdsService` potrebuje pristupovať k databáze cez `PrismaService`, stačí túto závislosť definovať v konštruktoze a NestJS sa postará o jej vytvorenie a prepojenie.

Tento prístup zvyšuje modularitu a umožňuje jednoduchú výmenu komponentov, čo je v súlade s princípmi čistého kódu (*Clean Code*).

4.2.3 Middleware a Guardy pre bezpečnosť

NestJS poskytuje robustný systém na spracovanie životného cyklu požiadavky (*Request Lifecycle*) (Casciaro a Mammino 2020). Každá prichádzajúca HTTP požiadavka prechádza presne definovanou cestou: najprv sa vykoná *Middleware*, následne *Guards* (strážcovia prístupu), *Interceptors* (zachytávače), *Pipes* (validácia a transformácia dát), až sa nakoniec dostane do samotného kontroléra a služby. Tento jasne štruktúrovaný tok umožňuje oddeliť biznis logiku od infraštruktúrnych a bezpečnostných úloh.

- **Guards (Strážcovia):** Využívam ich na ochranu endpointov. `JwtAuthGuard` overuje platnosť tokenu a `RolesGuard` kontroluje, či má používateľ rolu `ADMIN`.
- **Interceptors a Pipes:** Tieto nástroje slúžia na transformáciu dát (napr. úprava formátu odpovede) a ich validáciu. Vďaka `ValidationPipe` je zaručené, že do systému sa nedostanú nekorektné dáta, ktoré by mohli spôsobiť pád aplikácie alebo bezpečnostnú dieru.

4.2.4 Integrácia s Prisma ORM

NestJS sa výborne dopĺňa s nástrojom Prisma. V projekte som vytvoril dedikovaný `PrismaModule`, ktorý spracováva pripojenie k PostgreSQL databáze. Služba `PrismaService` priamo rozširuje triedu `PrismaClient` a využíva zabudované hooky životného cyklu NestJS, konkrétne `onModuleInit`. Vďaka tomu sa spojenie s databázou nadviaže hneď pri štarte aplikácie, čo zaručuje, že API nezačne prijímať požiadavky skôr, než je databáza plne pripravená.

Prisma funguje ako typovo bezpečný most (ORM), ktorý mi umožňuje pracovať s databázou pomocou bežných TypeScript objektov namiesto písania surových SQL dopytov.

Výsledkom je kód, ktorý je menej náchylný na chyby a oveľa rýchlejší sa vyvíja.

4.3 FRONTEND: NEXT.JS (APP ROUTER), REACT A TAILWIND CSS

Frontendová architektúra pozostáva z troch samostatných aplikácií (`platform`, `user`, `admin`), ktoré zdieľajú rovnaký technologický základ. Tento prístup umožňuje izolovať používateľské rozhrania podľa ich účelu, pričom sa zachováva konzistentný vývojový proces.

4.3.1 Next.js a architektúra App Router

Použitie Next.js (verzia 16+) prináša do projektu revolučný prístup k vykresľovaniu stránok prostredníctvom App Routera.

- **React Server Components (RSC):** Väčšina komponentov je predvolene vykresľovaná na strane servera. To znamená, že klient dostane hotový HTML dokument, čo dramaticky zrýchľuje prvé načítanie stránky a zlepšuje SEO, keďže vyhľadávače nemusia spúšťať zložitý JavaScript, aby videli obsah inzerátov. Okrem výkonu prinášajú RSC aj bezpečnostné výhody – umožňujú pristupovať k citlivým dátam alebo premenným prostredia (napr. API kľúčom) priamo na serveri bez toho, aby boli tieto údaje odoslané do prehliadača klienta.
- **Smerovanie (File-based Routing):** Next.js využíva adresárovú štruktúru na definovanie trás. Vďaka tomu je navigácia v aplikácii intuitívna a podporuje pokročilé funkcie ako paralelné trasy alebo interceptovanie trás (napr. zobrazenie detailu inzerátu v modálnom okne bez straty kontextu zoznamu).
- **Optimalizácia:** Framework automaticky optimalizuje obrázky, fonty a skripty, čo zabezpečuje plynulý chod aplikácie aj na slabších mobilných zariadeniach.

4.3.2 Deklaratívne UI s knižnicou React

Samotné používateľské rozhranie je postavené na knižnici React (Riva 2022). Využívam moderné koncepty ako Hooks (`useState`, `useEffect`, `useMemo`) na správu stavu a vedľajších účinkov.

- **Komponentový prístup:** Každý prvok rozhrania (tlačidlo, karta inzerátu, vyhľadávací panel) je samostatný znovupoužiteľný komponent. To uľahčuje údržbu kódu a zabezpečuje vizuálnu jednotnosť naprieč všetkými tromi aplikáciami v monorepe.

Príklad komponentu s využitím podmienkového renderovania a Tailwind CSS (ukážka z `CreateAdvertisementWizard`):


```
function CategoryVisualTile({ cat, selected, onSelect, tileVariant = 'platform'
}) {
  const isAdmin = tileVariant === 'admin';
  return (
    <button
      type="button"
      onClick={onSelect}
      className={
        isAdmin
          ? 'group flex flex-col items-center rounded-xl border-2 bg-card px-2 py
-5 text-center transition-all ${
            selected ? 'border-primary ring-2 ring-primary/25' : 'border-card
hover:border-primary/40'
          }'
          : 'group flex flex-col items-center rounded-2xl border-2 bg-white px-2
py-5 text-center transition-all ${
            selected ? 'border-[#1dbf73] ring-2 ring-[#1dbf73]/25' : '
border-white/80 hover:border-[#1dbf73]/40'
          }'
        }
    >
      <span className={isAdmin ? 'text-white' : 'text-gray-900'}>{cat.name}</span>
    </button>
  );
}
```

Táto ukážka demonštruje, ako jeden komponent dokáže dynamicky meniť svoj vzhľad na základe toho, či je vykresľovaný vo verejnej platforme alebo v administrátorskom rozhraní.

- **Klientska interaktivita a hydratácia:** Kým jadro stránky je serverové, interaktívne prvky (napr. chat, mapové filtre alebo formuláre) využívajú klientske komponenty (označené direktívou `'use client'`). Pri kombinácii týchto prístupov dochádza k procesu zvanému *hydratácia*. Next.js najprv pošle na klienta statické HTML pre rýchle zobrazenie obsahu a následne toto rozhranie „oživí“ pomocou JavaScriptu, čím sa dosiahne plná interaktivita a okamžitá odozva na akcie používateľa.

4.3.3 Styling pomocou Tailwind CSS

Na definovanie vizuálnej identity platformy som zvolil Tailwind CSS, čo je *utility-first* CSS framework (Riva 2022).

- **Rýchlosť vývoja a purging:** Namiesto písania tisícok riadkov v samostatných CSS súboroch definujem štýly priamo v HTML/JSEX pomocou predpripravených tried. Tailwind počas zostavovania (*build*) aplikácie analyzuje zdrojové súbory a do výsledného

CSS zahrnie iba tie triedy, ktoré sú skutočne použité. Tento proces (scanning/purging) eliminuje „mŕtvý kód“ (*dead code*) a výsledný štýl pre celú platformu má často veľkosť len niekoľko desiatok kilobajtov.

- **Responzivita a Mobile-first:** Tailwind natívne podporuje responzívny dizajn. Vďaka tomu bolo možné jednoducho implementovať rozhranie, ktoré sa automaticky prispôsobuje od malých displejov smartfónov až po veľké desktopové monitory.
- **Konzistencia:** Využitie konfiguračného súboru `tailwind.config.js` zabezpečuje, že farby, písma a rozostupy sú identické vo všetkých aplikáciách, čo buduje profesionálny dojem z platformy.

4.3.4 Integrácia s mapovými podkladmi (Leaflet)

Pre inzertný portál je kľúčová vizualizácia polohy. Vo fronte aplikácie `platform` využívam knižnicu Leaflet v kombinácii s React komponentmi. Táto integrácia umožňuje návštevníkom dynamicky prehliadať inzeráty na interaktívnej mape, pričom dáta o polohe (latitúda a longitúda) sú v reálnom čase sťahované z NestJS API.

Keďže knižnice ako Leaflet pracujú priamo s objektom `window`, ktorý na serveri neexistuje, bolo nutné vyriešiť kompatibilitu so Server-Side Renderingom (SSR). V Next.js sa takéto klientske knižnice načítavajú asynchrónne pomocou funkcie `dynamic` z modulu `next/dynamic`, typicky zápisom v tvare:

```
dynamic(() => import(...), { ssr: false })
```

Toto riešenie zabezpečí, že sa mapa inicializuje až priamo v prehliadači používateľa, čím sa predíde chybám pri renderovaní na serveri.

4.4 PERZISTENCIA DÁT: POSTGRESQL A PRISMA ORM

Efektívna správa dát je základom každej úspešnej webovej aplikácie. V mojom projekte som sa rozhodol pre technologický zásobník, ktorý zaručuje integritu údajov, vysoký výkon pri zložitých dopytoch a vynikajúcu skúsenosť pri vývoji v TypeScript.

4.4.1 Relačná databáza PostgreSQL

Ako úložisko dát som zvolil PostgreSQL, čo je pokročilá open-source objektovo-relačná databáza (Obe a Hsu 2017).

- **Spôľahlivosť a integrita:** PostgreSQL striktne dodržiava princípy ACID (*Atomicity, Consistency, Isolation, Durability*) (Obe a Hsu 2017), čo je nevyhnutné pri operáciách, ako je registrácia používateľa alebo spracovanie správ, kde nesmie dôjsť k strate či poškodeniu dát.

- **Podpora geografických dát:** Vzhľadom na to, že platforma využíva mapové podklady, PostgreSQL je ideálnou voľbou vďaka svojej schopnosti efektívne pracovať so súradnicami a priestorovými dopytmi.
- **Výkon pri vyhľadávaní:** Databáza umožňuje vytvárať indexy nad kľúčovými poľami (napr. `slug`, `categoryId`, `createdAt`), čo zabezpečuje bleskovú odozvu pri filtrovaní tisícok inzerátov.
- **Flexibilita s JSONB:** Využitie typu JSONB v PostgreSQL mi umožňuje ukladať dynamické parametre inzerátov (špecifikácie) s vysokým výkonom, pričom nad týmito dátami je možné vykonávať efektívne dopyty takmer rovnako rýchlo ako nad klasickými stĺpcami.

4.4.2 Prisma ORM (Object-Relational Mapping)

Na komunikáciu medzi NestJS backendom a databázou PostgreSQL využívam Prisma ORM (Winters et al. 2020). Na rozdiel od tradičných ORM nástrojov, Prisma funguje na báze deklaratívnej schémy, ktorá slúži ako jediný zdroj pravdy (*Single Source of Truth*) pre celý projekt.

- **Typová bezpečnosť (*Type-safety*):** Prisma automaticky generuje TypeScript klient na základe definovanej schémy (`schema.prisma`). Výsledný klient je dostupný ako npm balík v priečinku `node_modules`. Vďaka tomu vývojové prostredie (napr. VS Code) presne vie, aké polia má napríklad model `User`, bez nutnosti ručného písania TypeScript rozhraní pre databázu. Pri písaní databázových dopytov je k dispozícii kompletne automatické dopĺňanie kódu a kompilátor upozorní na akúkoľvek nezrovnalosť.
- **Prisma Schema:** V tomto súbore definujem nielen tabuľky (modely), ale aj ich vzájomné vzťahy (napr. `@relation` medzi inzerátom a autorom) a enumerátory (`enum`) pre stavy inzerátov či roly používateľov.
- **Migrácie (Prisma Migrate):** Každú zmenu v štruktúre databázy spravujem pomocou migrácií. Prisma sleduje históriu zmien a umožňuje ich bezpečné a reprodukovateľné nasadenie na produkčný server, čím sa eliminuje riziko manuálnych chýb pri správe SQL skriptov.
- **Výkon a optimalizácia (SQL vs. ORM):** Hoci použitie ORM môže v porovnaní s čistým SQL priniesť mierny režijný náklad (*overhead*), Prisma je vysoko optimalizovaná a generuje efektívne SQL dopyty. Tento minimálny rozdiel vo výkone je bohato kompenzovaný elimináciou ľudských chýb pri písaní dopytov a obrovským zrýchlením vývoja vďaka typovej bezpečnosti.

4.4.3 Prospech pre Monorepo architektúru

V rámci monorepa je Prisma umiestnená v samostatnom balíku `packages/database`. Tento prístup umožňuje:

- **Zdieľanie schémy:** API aplikácia využíva vygenerovaný klient na zápis a čítanie dát, zatiaľ čo ostatné časti projektu môžu využívať vygenerované typy pre validáciu.
- **Jednoduchá údržba:** Ak potrebujem pridať nové pole do inzerátu (napríklad „vratná kaucia“), zmením to na jednom mieste v schéme, spustím migráciu a zmena sa okamžite prejaví v celom vývojovom prostredí.

4.5 VOL'BA TECHNOLOGIÍ: NODE.JS EKOSYSTÉM VERZUS PHP

Pri rozhodovaní o technologickom zásobníku som zvažoval dve hlavné alternatívy: klasický prístup postavený na PHP (napríklad frameworky Laravel alebo Symfony) a moderný prístup postavený na Node.js s TypeScriptom (NestJS na backende, Next.js na frontende). Táto kapitola popisuje dôvody, prečo som sa rozhodol pre druhú možnosť.

4.5.1 Asynchrónny model spracovania požiadaviek

PHP tradične funguje na modeli *one-process-per-request*: každá HTTP požiadavka spustí nový PHP proces (alebo vlákno), ktorý blokuje výpočtové zdroje počas celého trvania operácie. Pri paralelnom prístupe tisícov používateľov to vedie k výraznému zaťaženiu servera.

Node.js naproti tomu využíva *event loop* – neblokujúci, jednovláknový model. Asynchrónne I/O operácie (databázové dopyty, čítanie súborov) sa vykonávajú na pozadí a vlákno sa uvoľní pre ďalšie požiadavky okamžite po spustení operácie. Pre inzertný portál, kde dominujú I/O operácie (čítanie inzerátov z databázy, odosielanie správ), je tento model ideálny a umožňuje obsluhovať oveľa väčší počet súbežných požiadaviek s nižšími hardvérovými nárokmi (Casciaro a Mammino 2020).

4.5.2 Jednotný jazyk naprieč celým stackom

Jednou z najvýznamnejších výhod zvoleného riešenia je použitie jediného jazyka – TypeScriptu – na backende aj frontende. PHP je výhradne serverový jazyk; frontend musí byť napísaný oddelene v JavaScripte. Výsledkom sú dve nezávislé kódové základne, ktoré si nemôžu zdieľať typy ani validačnú logiku.

V mojom monorepe je balík `packages/shared` zdieľaný medzi NestJS API a všetkými tromi Next.js aplikáciami. Ak zmením štruktúru DTO pre inzerát, TypeScript kompilátor okamžite upozorní na nezrovnalosti na všetkých miestach – na backende aj frontende – ešte pred spustením aplikácie. Tento mechanizmus *end-to-end* typovej bezpečnosti (Cherny 2019) prakticky eliminuje celú kategóriu chýb spôsobených nekonzistentnými dátovými typmi, ktoré sú v PHP/JavaScript projektoch bežné.

4.5.3 Server-Side Rendering a SEO

V PHP je generovanie HTML na serveri prirodzenou súčasťou jazyka (šablóny Blade, Twig, Latte). Avšak moderné PHP aplikácie stále čoraz viac spoliehajú na JavaScript SPA (Single Page Application), čím strácajú výhodu natívneho SSR a vyžadujú komplexné konfigurácie na zachovanie SEO.

Next.js prináša to najlepšie z oboch svetov: React Server Components generujú HTML priamo na serveri (rovnako ako PHP), zatiaľ čo interaktívne časti stránky sú hydratované v prehliadači. Pre inzertný portál je toto kritické – vyhľadávače ako Google musia vedieť indexovať obsah inzerátov okamžite z HTML, bez nutnosti spúšťania JavaScriptu. Zároveň Next.js automaticky optimalizuje obrázky, fonty a skriptové súbory, čo by v PHP vyžadovalo manuálnu konfiguráciu ďalších nástrojov (Riva 2022).

4.5.4 Ekosystém, nástroje a Developer Experience

PHP ekosystém (Composer, PHPUnit, Eloquent ORM) je síce zrelý a stabilný, avšak npm ekosystém je oveľa rozsiahlejší. Pre kľúčové komponenty mojej platformy – interaktívne mapy (Leaflet), vizuálny editor obsahu (GrapesJS), real-time chat – existujú natívne npm balíky s aktuálnou podporou. Integrácia rovnakých knižníc do PHP backendu by si vyžiadala ich použitie len na frontende, čo zvyšuje komplexitu projektovej štruktúry.

Prisma ORM pre Node.js poskytuje typovo bezpečné databázové dopyty s autokompletizáciou v IDE, ktorá nemá priamu obdobu v PHP. Eloquent ORM v Laraveli je síce elegantný, ale neposkytuje kompilačnú typovú bezpečnosť; chyby v databázových dopytoch sa prejavajú až pri behu aplikácie.

4.5.5 Zhrnutie porovnania

Tabuľka 2 sumarizuje kľúčové rozdiely medzi oboma prístupmi z pohľadu požiadaviek tohto projektu.

Kritérium	PHP (Laravel)	Node.js / TypeScript
Model požiadaviek	Blokujúci (process-per-request)	Neblokujúci (event loop)
Zdieľanie typov	Nie (2 jazyky)	Áno (monorepo, TS)
SSR + SEO	Šablónový engine alebo SPA+konfigurácia	Natívne (Next.js RSC)
Typová bezpečnosť	Čiastočná (PHP 8+)	Kompletná (TypeScript)
ORM	Eloquent (runtime chyby)	Prisma (compile-time)
Mapové knižnice	JS knižnice cez npm (frontend only)	Priama integrácia (React)
Nasadenie	Apache/Nginx + PHP-FPM	Node.js / Docker / Vercel

Tabuľka 2: Porovnanie PHP (Laravel) a Node.js/TypeScript ekosystému

Na základe tohto porovnania je zrejmé, že pre projekt s troma frontendovými aplikáciami, zdieľanými typmi a požiadavkou na SSR je Node.js/TypeScript stack výrazne vhodnejší ako tradičné PHP riešenie.

4.6 NÁSTROJE PRE SPRÁVU KÓDU A KONTAJNERIZÁCIU (GIT, DOCKER COMPOSE)

Efektívny vývoj softvéru vyžaduje nielen kvalitný kód, ale aj spoľahlivé nástroje na jeho verzovanie a orchestráciu služieb. V mojom projekte hrajú kľúčovú úlohu systémy Git a Docker, ktoré umožňujú bezpečnú správu verzií a izoláciu behového prostredia.

4.6.1 Verzovanie kódu pomocou systému Git

Systém Git je základným nástrojom na sledovanie zmien v zdrojovom kóde (Winters et al. 2020). V kontexte monorepa je jeho dôležitosť ešte vyššia, nakoľko spravuje históriu viacerých aplikácií a zdieľaných balíkov súčasne.

- **Vetvenie (*Branching strategy*):** Pri vývoji využívam logické rozdelenie funkcií do samostatných vetiev (napr. `feature/auth`, `feature/map-logic`). To umožňuje experimentovať s novými funkciami bez rizika narušenia stability hlavnej vetvy (`main`).
- **História a audit:** Git uchováva detailný prehľad o tom, kedy a prečo bola konkrétna zmena vykonaná. To je kľúčové pri hľadaní chýb (*debugging*) alebo pri návrate k predchádzajúcej stabilnej verzii aplikácie.
- **Ignorovanie súborov (`.gitignore`):** Pomocou tohto súboru zabezpečujem, že sa do repozitára nedostanú zbytočné dáta (napríklad vygenerované priečinky `node_modules`) alebo citlivé údaje (ako sú konfiguračné súbory `.env`).

4.6.2 Kontajnerizácia pomocou technológie Docker

Pre zabezpečenie konzistentného správania aplikácie som sa rozhodol pre Docker (Winters et al. 2020). Táto technológia umožňuje „zabalit“ aplikáciu spolu so všetkými jej závislosťami (Node.js, knižnice, konfiguračné súbory) do izolovaného kontajnera.

- **Docker Image:** Každá časť systému (API, frontendy) má definovaný svoj vlastný `Dockerfile`. Ten obsahuje inštrukcie na zostavenie obrazu, čím sa eliminuje problém typu „u mňa v počítači to funguje“, pretože kontajner beží v identickom prostredí bez ohľadu na operačný systém hostiteľa.

- **Izolácia databázy:** Docker využívam aj na spustenie databázy PostgreSQL. Vďaka tomu nemusím databázový server inštalovať priamo do operačného systému, čo udržiava vývojové prostredie čisté a ľahko prenosné.

4.6.3 Orchestrácia služieb cez Docker Compose

Vzhľadom na to, že môj projekt pozostáva z viacerých súbežne bežiacich služieb (API, tri frontendy a databáza), využívam nástroj Docker Compose.

- **Konfigurácia jedným súborom:** Súbor `docker-compose.yml` definuje celý ekosystém aplikácie. Obsahuje informácie o portoch, prepojeniach medzi kontajnermi a premenných prostredia.
- **Jednoduchosť spustenia:** Celé komplexné prostredie je možné uviesť do prevádzky jediným príkazom `docker-compose up`. Tento prístup dramaticky zrýchľuje proces nasadenia (*deployment*) a uľahčuje spoluprácu, ak by sa na projekte podieľalo viacero vývojárov.
- **Bezpečnosť a siete (*Networking*):** Vďaka Docker Compose sú služby ako API a databáza prepojené v spoločnej internej virtuálnej sieti. To zvyšuje bezpečnosť, pretože databázový port nemusí byť vystavený do verejného internetu, ale je prístupný výhradne pre backendovú aplikáciu.

4.6.4 Prepojenie s monorepom a optimalizácia obrazov

Nástroje na správu kódu sú plne integrované s `npm workspaces`. Pri vytváraní Docker obrazov využívam viacúrovňové zostavovanie (*multi-stage builds*). V prvej fáze (*build stage*) sa nainštalujú všetky vývojárske závislosti a TypeScript kód sa skompiluje. V druhej fáze (*production stage*) sa do výsledného obrazu prekopíruje už len hotový JavaScript a produkčné knižnice (vrátane nevyhnutných súborov zo zdieľaných balíkov `packages/`).

Výsledný kontajner je tak o stovky megabajtov menší a neobsahuje pôvodný zdrojový kód, čo sťažuje prípadný útok. K bezpečnosti a optimalizácii veľkosti prispieva aj súbor `.dockerignore`, ktorý zabráňuje prekopírovaniu zbytočností (ako lokálne `node_modules`) či citlivých dát (`.env` súbory) do vnútra kontajnera pri jeho zostavovaní.

5 REALIZÁCIA A IMPLEMENTÁCIA SYSTÉMU

Implementácia systému prebiehala v súlade s princípmi moderného full-stack vývoja. V tejto kapitole detailne rozoberám vnútornú štruktúru projektu, spôsob zdieľania kódu a konkrétne technické riešenia jednotlivých modulov.

5.1 ŠTRUKTÚRA MONOREPA A ZDIEĽANIE TYPOV

Pre správu celého ekosystému som zvolil architektúru monorepa s využitím nástroja *npm workspaces* (Winters et al. 2020). Tento prístup mi umožnil sústrediť frontendovú aj backendovú logiku do jedného úložiska, čo výrazne zjednodušilo refaktoring a správu závislostí.

5.1.1 Organizácia adresárovej štruktúry

Projekt je rozdelený do dvoch hlavných priečinkov, ktoré jasne oddelujú spustiteľné aplikácie od knižníc. Pre lepšiu orientáciu uvádzam zjednodušenú štruktúru projektu:

```
.
|-- apps/
|   |-- admin/          # Next.js (Admin panel)
|   |-- api/            # NestJS (Backend)
|   |-- platform/       # Next.js (Verejny web)
|   '-- user/           # Next.js (Pouzivatelska zona)
|-- packages/
|   |-- database/       # Prisma schema a klient
|   '-- shared/         # DTOs, TypeScript typy
|-- package.json        # npm workspaces
'-- docker-compose.yml
```

- **apps/**: Obsahuje štyri samostatné projekty: **api** (NestJS), **platform** (verejný Next.js web), **user** (používateľská zóna) a **admin** (správcovský panel). Každá z týchto aplikácií má vlastnú konfiguráciu a beží na samostatnom porte.
- **packages/**: Obsahuje zdieľané moduly, ktoré nie sú samostatne spustiteľné, ale poskytujú logiku pre ostatné aplikácie. Najdôležitejšími sú **database** (Prisma schéma a klient) a **shared** (spoločné typy).

5.1.2 Implementácia packages/shared a zdieľanie typov

Jednou z najväčších výziev pri vývoji oddeleného frontendu a backendu je udržanie konzistentných dátových typov. V mojom projekte som tento problém vyriešil vytvorením balíka **shared**, ktorý slúži ako „jediný zdroj pravdy“.

- **Zdieľané rozhrania (Interfaces)**: V tomto balíku definujem štruktúru všetkých objektov, ktoré prechádzajú cez sieť. Ak napríklad definujem rozhranie **IAdvertisement**,

obe strany (API aj frontend) presne vedia, aké polia má daný objekt obsahovať (napr. `title`, `price`, `images`).

- **DTO (*Data Transfer Objects*):** Pomocou knižnice `class-validator` definujem v tomto balíku aj pravidlá pre validáciu vstupov. Tieto DTO triedy využíva NestJS na kontrolu prichádzajúcich požiadaviek a zároveň ich Next.js využíva na definovanie typov vo formulároch. Je dôležité zdôrazniť rozdelenie zodpovednosti: kým TypeScript stráži typy počas vývoja (*compile-time*), `class-validator` stráži dáta počas behu aplikácie (*runtime*) a overuje napríklad to, či je zadaný textový reťazec skutočne platným e-mailom.
- **End-to-End bezpečnosť:** Vďaka tomuto prepojeniu ma kompilátor TypeScriptu okamžite upozorní, ak na backende zmením názov poľa v databáze, ale na frontende sa ho stále pokúšam zobrazit'. Týmto spôsobom som eliminoval veľké množstvo chýb, ktoré bežne vznikajú pri manuálnom prepisovaní typov.

5.1.3 Mechanizmus npm workspaces

Nástroj *npm workspaces* zabezpečuje, že všetky balíky v rámci monorepa sú navzájom prepojené prostredníctvom symbolických odkazov (*symlinks*). To znamená, že pri lokálnom vývoji nemusím balík `shared` zakaždým publikovať; zmeny vykonané v zdieľanom kóde sa okamžite prejavia vo všetkých aplikáciách, ktoré ho importujú. Tento mechanizmus je kľúčový pre agilný vývoj a rýchle prototypovanie nových funkcií.

Okrem toho *npm workspaces* rieši problém s duplicitnými závislosťami (tzv. *hoisting*). Všetky aplikácie zdieľajú jeden spoločný priečinok `node_modules` v koreni projektu, čo výrazne šetrí miesto na disku a zrýchľuje inštaláciu závislostí.

5.2 REALIZÁCIA BACKENDOVÝCH MODULOV

Každý modul v systéme pozostáva z troch základných vrstiev: *Controller* (vstupná brána a definícia endpointov), *Service* (aplikačná logika a komunikácia s databázou) a *Module* (konfigurácia závislostí) (Martin 2017).

5.2.1 Modul Auth: Autentifikácia a autorizácia

Tento modul je zodpovedný za bezpečnosť a správu identít. Implementácia využíva knižnicu `@nestjs/passport` a stratégie pre JWT.

- **Registrácia a login:** Služba `AuthService` využíva `bcrypt` na porovnávanie hashovaných hesiel. Pri úspešnom prihlásení vygeneruje bezpečne podpísaný JWT token s exspiráciou. Payload tohto tokenu obsahuje identifikátor používateľa

(`sub`) a jeho rolu (`role`), čo minimalizuje nutnosť dopytovať sa do databázy pri každom overovaní oprávnení.

- **Zabezpečenie:** Na ochranu endpointov som vytvoril globálne guardy. `JwtAuthGuard` extrahuje token z hlavičky HTTP požiadavky a validuje ho, zatiaľ čo `RolesGuard` v spolupráci s dekorátorom `@Roles()` kontroluje úroveň oprávnení (napr. prístup k mazaniu používateľov).

5.2.2 Modul Ads: Správa inzerátov a médií

Modul `AdsModule` je najrozsiahlejšou časťou backendu, nakoľko spracováva hlavnú entitu systému – inzerát.

- **CRUD operácie:** Implementoval som endpointy pre vytvorenie, čítanie, aktualizáciu a mazanie inzerátov. Pri vytváraní inzerátu sa automaticky generuje unikátny slug z názvu pre SEO účely.
- **Spracovanie polohy:** Inzeráty ukladajú GPS súradnice (latitúda, longituda), ktoré sú nevyhnutné pre zobrazenie na mape vo frontende.
- **Stavy inzerátov:** Logika v `AdsService` zabezpečuje prechody medzi stavmi `PENDING` (čakajúci na schválenie), `ACTIVE` (viditeľný) a `ARCHIVED`.

Ukážka schvaľovacieho procesu inzerátu z `AdvertisementsService`:

```

async approve(id: string) {
  const advertisement = await prisma.advertisement.findUnique({
    where: { id } });

  if (!advertisement) throw new NotFoundException('Inzerát nebol
    nájdený');
  if (advertisement.status !== 'PENDING') throw new
    ForbiddenException('Inzerát už bol spracovaný');

  const updated = await prisma.advertisement.update({
    where: { id },
    data: { status: 'ACTIVE' },
  });

  // systémová správa o schválení pre používateľa
  await this.messagesService.createSystemMessage(
    advertisement.userId,
    'AD_APPROVED',
    'Váš inzerát bol schválený',
    'Váš inzerát "${advertisement.title}" bol schválený a je teraz
      aktívny na platforme.',
    { advertisementId: id }
  );

  return updated;
}

```

Táto ukážka jasne demonštruje, ako je služba napísaná odolne voči chybám (využitím `NotFoundException` a `ForbiddenException`). Kombinuje validáciu stavu, aktualizáciu v databáze a komunikáciu s iným modulom (`MessagesService`) pre odoslanie notifikácie používateľovi ako vedľajší efekt.

5.2.3 Modul Categories a dynamické filtre

Tento modul spravuje hierarchickú štruktúru portálu.

- **Stromová štruktúra:** Kategórie podporujú rekurzívne vnorenie (Parent-Child). API vracajú stromovú štruktúru dát, ktorá sa využíva v navigačnom menu platformy.
- **Logika filtrov:** Každá kategória môže mať pridelenú sadu filtrov. Služba zabezpečuje, že pri vyhľadávaní v kategórii „Autá“ sa vrátia inzeráty filtrované podľa špecifických polí uložených v JSON objekte v databáze. Využívam tu schopnosť Prisma ORM dopytovať sa priamo do vnútra `JSONB` stĺpcov, čím sa zachováva vysoký výkon vyhľadávania aj pri neštruktúrovaných dátach.

5.2.4 Modul Search: Full-textové vyhľadávanie a analytika

Pre efektívne vyhľadávanie som implementoval dedikovanú logiku, ktorá kombinuje viaceré parametre.

- **Filtrovanie:** Vyhľadávanie prebieha kombináciou full-textového hľadania v názve a popise inzerátu s presným filtrovaním podľa kategórie, ceny a lokality.
- **ClickEvent integrácia:** Každé zobrazenie detailu inzerátu vyvolá asynchrónne volanie v `AnalyticsService`, ktoré zaznamená udalosť `ClickEvent`. Vďaka asynchrónnemu spracovaniu „na pozadí“ používateľ vôbec nečaká na zapísanie štatistiky do databázy a inzerát sa mu zobrazí okamžite. Tieto dáta sa ukladajú s informáciou o type používateľa (pohlavie, vek), čo neskôr slúži na generovanie štatistík v admin paneli.

5.3 VÝVOJ KLIENTSKÝCH APLIKÁCIÍ

Všetky tri frontendové aplikácie (`platform`, `user`, `admin`) sú vybudované na rovnakom základe: Next.js App Router s React Server Components, Tailwind CSS a typmi zo zdieľaného balíka `packages/shared`. Napriek spoločnému technologickému základu má každá aplikácia odlišný účel, vizuálnu identitu a sadu komponentov prispôbených konkrétnej role používateľa (Riva 2022).

5.3.1 Verejná platforma (vyhľadávanie, mapa, blog)

Aplikácia `apps/platform` je vstupnou bránou celej platformy a je optimalizovaná predovšetkým na rýchlosť a SEO. Väčšina stránok (zoznamy inzerátov, detail inzerátu, blog, statické stránky) je vykresľovaná ako React Server Components, čo zaručuje okamžitú odozvu a plnú indexovateľnosť obsahu vyhľadávačmi.

Vyhľadávanie a filtrovanie: Kľúčovým prvkom je vyhľadávací panel, ktorý dynamicky načítava filtre podľa zvolenej kategórie z API. Parametre vyhľadávania sú zakódované priamo do URL adresy (`searchParams`), čo umožňuje zdieľanie odkazu na konkrétny výsledok hľadania a zároveň zaručuje správnu indexáciu výsledkov vyhľadávačmi.

Mapové zobrazenie (Leaflet): Stránka s mapou zobrazuje všetky aktívne inzeráty ako interaktívne markery. Dáta o polohe (latitúda, longitúda) sú načítané z endpointu `GET /advertisements/map`. Keďže knižnica Leaflet pristupuje k objektu `window`, ktorý na serveri neexistuje, komponent mapy je načítaný asynchrónne pomocou `next/dynamic` s parametrom `ssr: false`, čím sa predíde hydratačným chybám.

Blog a statické stránky: Obsah blogu a informačných stránok je spravovaný cez administratívny panel a uložený v databáze ako HTML. Platforma ho zobrazuje vo forme staticky renderovaných stránok, ktoré sú automaticky generované pri každom requeste (SSR) alebo môžu byť cachované (ISR), čo minimalizuje záťaž na databázu.

5.3.2 Používateľská zóna (správa inzerátov a profilu)

Aplikácia `apps/user` je dostupná výhradne pre prihlásených používateľov. Autentifikácia je riešená prostredníctvom JWT tokenu, ktorý je po prihlásení uložený v HTTP-only cookie, čím je chránený pred útokmi XSS (Hoffman 2020).

Sprievodca tvorbou inzerátu (Wizard): Namiesto jednoduchého formulára som implementoval viacstupňového sprievodcu (*wizard*), ktorý vedie používateľa krok za krokom: výber kategórie → zadanie detailov a špecifikácií → nahranie fotografií → zadanie polohy na mape → odoslanie na schválenie. Tento prístup výrazne znižuje kognitívnu záťaž pri tvorbe inzerátu a prispieva k vyššej kvalite zadaných dát.

Správa inzerátov: Dashboard používateľa zobrazuje všetky jeho inzeráty vrátane ich aktuálneho stavu (`PENDING`, `ACTIVE`, `ARCHIVED`) a štatistík zobrazení. Používateľ môže inzeráty editovať, deaktivovať alebo zmazať.

Komunikácia – Inbox: Modul správ zobrazuje zoznam konverzácií (dopytov k inzerátom) a umožňuje odpovedanie v reálnom čase. Systémové notifikácie (napr. o schválení inzerátu) sú vizuálne odlíšené od bežných správ od iných používateľov.

Oblíbené položky: Stránka zobrazuje inzeráty uložené používateľom do oblíbených. Pridávanie a odoberanie z oblíbených je realizované optimistickým UI updatem – tlačidlo reaguje okamžite bez čakania na odpoveď servera, čo zvyšuje vnímaný výkon.

5.3.3 Administračný panel (CMS, vizuálny editor GrapesJS, štatistiky)

Aplikácia `apps/admin` poskytuje správcovi platformy kompletné nástroje na správu obsahu, používateľov a konfigurácii systému. Je chránená dvojvrstvovým overovaním: platným JWT tokenom a rolou `ADMIN`.

Moderácia inzerátov: Hlavnou obrazovkou admina je zoznam inzerátov čakajúcich na schválenie (`PENDING`). Pre každý inzerát zobrazuje náhľad, fotografie a lokalizáciu na mape. Administrátor môže inzerát schváliť (zmena stavu na `ACTIVE`), zamietnuť alebo archivovať. Každá akcia automaticky odošle systémovú správu inzerentovi.

Vizuálny editor GrapesJS: Na správu statických stránok a blogových príspevkov som integroval open-source editor GrapesJS. Umožňuje administrátorovi vytvárať a upravovať HTML obsah stránok drag-and-drop spôsobom bez znalosti HTML kódu. Výsledný HTML kód je uložený do databázy a zobrazovaný na verejnej platforme. Aj tu bolo potrebné riešiť kompatibilitu s SSR pomocou asynchrónneho načítavania modulu cez `next/dynamic` s voľbou `ssr: false`, ktorá zaručuje inicializáciu editora výhradne v prehliadači.

Správa kategórií a filtrov: Admin panel umožňuje vytvárať a editovať hierarchiu kategórií (hlavné kategórie, podkategórie) a priradiť im dynamické filtre. Každý filter má definovaný typ (`TEXT`, `NUMBER`, `SELECT`), názov a poradie zobrazenia. Zmeny sa okamžite

premietnu do vyhl'adavacieho panela na verejnej platforme bez nutnosti nasadenia novej verzie aplikácie.

Analytický dashboard: Panel štatistík zobrazuje grafy kliknutí na inzeráty v čase, rozdelenie záujmu podľa kategórií a segmentáciu podľa typu používateľ'a. Dáta pochádzajú z tabuľky `ClickEvent`, ktorá je plnená asynchrónne pri každom zobrazení detailu inzerátu (Winters et al. 2020).

5.4 BEZPEČNOSTNÉ ASPEKTY: HASHOVANIE HESIEL (BCRYPT), VALIDÁCIA VSTUPOV (CLASS-VALIDATOR) A OCHRANA ÚČTOV (BANOVANIE)

Bezpečnosť je prierezovou vlastnosťou celého systému. Navrhnuté opatrenia pokrývajú tri hlavné oblasti: ochranu hesiel, integritu vstupných dát a správu používateľ'ských účtov (Hoffman 2020).

Hashovanie hesiel (bcrypt): Heslá používateľ'ov nie sú nikdy ukladané v čitateľnej podobe. Pred uložením sú transformované pomocou adaptívneho hašovacieho algoritmu `bcrypt` s hodnotou `saltRounds = 10` (Hoffman 2020). `Bcrypt` je úmyselne pomalý – sť'ažuje útoky hrubou silou (*brute-force*) a dúhové tabuľky (*rainbow tables*). Pri prihlásení sa zadané heslo porovná s uloženým hašom pomocou funkcie `bcrypt.compare()`, pričom pôvodné heslo nikdy neopúšťa pamäť servera.

Validácia vstupov (class-validator): Každá prichádzajúca HTTP požiadavka prechádza validáciou pomocou globálne nastaveného `ValidationPipe` v konfigurácii `NestJS` (Hoffman 2020). Pravidlá validácie sú deklarované pomocou dekorátorov z knižnice `class-validator` priamo v DTO triedach – napríklad `@IsEmail()`, `@IsString()` alebo `@MinLength(8)`. Nastavenie `whitelist: true` spolu s `forbidNonWhitelisted: true` zaručuje, že akékoľvek polia navyše, ktoré nie sú definované v DTO, sú automaticky odmietnuté. Toto opatrenie bráni tzv. *mass assignment* útokom, pri ktorých útočník pošle nepovolené pole (napr. `role: "ADMIN"`) s cieľom zmeniť citlivé atribúty objektu.

Ochrana účtov – banovanie: Administrátor môže používateľ'a zablokovat' dočasne (pole `bannedUntil`) alebo trvalo (pole `banned: true`). Dôvod blokovania je uložený v poli `banReason`. Pri každom prihlásení aj pri spracovaní autorizovaných požiadaviek `JwtAuthGuard` overí stav banu a v prípade aktívneho blokovania vráti odpoveď `403 Forbidden`. Tento mechanizmus zabezpečuje, že zablokováný používateľ stratí prístup k systému aj bez nutnosti invalidovania jeho existujúcich tokenov.

5.5 SYSTÉM DYNAMICKÝCH FILTROV A KATEGÓRIÍ

Jednou z kľúčových architektonických výziev inzertného portálu je riešenie heterogénnej povahy inzerátov: prenájom auta má iné atribúty (rok výroby, počet km) ako inzerát na opravu počítačov (typ zariadenia, cena za hodinu). Namiesto vytvárania špecializovaných

databázových tabuliek pre každú kategóriu som zvolil flexibilný prístup s dynamickými filrami.

Architektúra filtrov: Každá kategória v systéme môže mať priradenú sadu `Filter` entít. Každý filter definuje `name` (názov atribútu), `type` (typ vstupného poľa: `TEXT`, `NUMBER`, `SELECT`, `MULTISELECT`) a prípadne `options` (povolené hodnoty pre `SELECT` typ). Inzerát ukladá hodnoty týchto parametrov do stĺpca `specifications` vo formáte JSONB v PostgreSQL, čo umožňuje flexibilné rozšírenie bez zmeny databázovej schémy.

Dynamické vyhľadávanie: Pri filtrovaní inzerátov API endpoint prijíma hodnoty filtrov ako URL parametre a zostavuje Prisma dopyt, ktorý prehľadáva JSONB stĺpec. PostgreSQL poskytuje natívnu podporu pre indexovanie JSONB polí pomocou operátorov `@>` (obsahuje) a funkcie `jsonb_path_exists`, čo zaručuje výkonné vyhľadávanie aj pri veľkom počte inzerátov (Obe a Hsu 2017).

Prínosy pre rozšíriteľnosť: Keď administrátor vytvorí novú kategóriu a priradí jej filtre, zmena sa okamžite prejaví na celej platforme – vyhľadávací panel automaticky zobrazí nové filtre bez nutnosti zásahu do kódu. Tento princíp tzv. *data-driven UI* je jedným z hlavných rozdielov oproti staticky nakódovaným filtrom, ktoré sú typické pre staršie portály ako Bazoš.

6 TESTOVANIE A VÝSLEDKY

Overenie správnosti systému prebiehalo na viacerých úrovniach: manuálnym testovaním API endpointov prostredníctvom Swagger dokumentácie, sledovaním analytických dát a vyhodnotením používateľského rozhrania na rôznych zariadeniach. Tento viacúrovňový prístup k testovaniu je v súlade so štandardnými praktikami softvérového inžinierstva (Sommerville 2015).

6.1 OVERENIE FUNKČNOSTI API (SWAGGER DOKUMENTÁCIA)

NestJS v kombinácii s balíkom `@nestjs/swagger` automaticky generuje interaktívnu API dokumentáciu prístupnú na adrese `GET /api/docs`. Swagger UI umožňuje volať každý endpoint priamo z prehliadača – zadávať vstupné parametre, odosielať autorizované požiadavky s JWT tokenom a okamžite overovať odpovede.

Pri testovaní som postupoval scenárovo – simuloval som reálne používateľské toky (Sommerville 2015):

1. **Registrácia a prihlásenie:** Overenie, že `POST /auth/register` vytvorí používateľa a `POST /auth/login` vráti platný JWT token.
2. **Tvorba inzerátu:** Odoslanie `POST /advertisements` s platným tokenom a overenie, že inzerát je vytvorený so stavom `PENDING`.
3. **Schválenie adminom:** Zavolanie `PATCH /advertisements/:id/approve` s admin tokenom a overenie zmeny stavu na `ACTIVE`.
4. **Ochrana endpointov:** Overenie, že volanie chránených endpointov bez tokenu vracia `401 Unauthorized` a volanie admin endpointov s bežným user tokenom vracia `403 Forbidden`.
5. **Validácia vstupov:** Odoslanie požiadaviek s chýbajúcimi alebo nesprávnymi poliami a overenie, že API vracia `400 Bad Request` s popisom chyby.

Všetky testované scenáre vrátili očakávané výsledky, čo potvrdilo správnosť implementácie validácie, autentifikácie aj biznis logiky.

6.2 ANALYTIKA A MONITORING (CLICKEVENT TRACKING A SEGMENTÁCIA POUŽÍVATEĽOV)

Analytický subsystém bol navrhnutý tak, aby generoval hodnotné dáta pre administrátora bez negatívneho vplyvu na výkon platformy. Každé zobrazenie detailu inzerátu asynchrónne

zaznamená udalosť `ClickEvent` do databázy – volanie `AnalyticsService` prebieha na pozadí a používateľ nečaká na jeho dokončenie.

Zaznamenávané atribúty zahŕňajú: identifikátor inzerátu, identifikátor kategórie, typ udalosti (`eventType`), pohlavie a typ účtu volajúceho používateľ a časovú pečiatku. Tieto dáta sú v admin paneli vizualizované ako:

- **Graf kliknutí v čase** – zobrazuje denný počet interakcií za posledných 30 dní, čo odhaľuje výkyvy návštevnosti.
- **Top kategórie** – rebríček kategórií podľa počtu kliknutí, ktorý pomáha administrátorovi rozhodnúť, ktoré kategórie treba rozšíriť.
- **Segmentácia podľa typu účtu** – pomer kliknutí od bežných používateľov vs. firemných účtov, čo indikuje, aký typ publika platforma priťahuje.

Výsledky analýzy ukázali, že tento asynchrónny prístup pridáva k celkovému času odozvy menej ako 5 ms, čo je z pohľadu používateľ a nemerateľný rozdiel.

6.3 VYHODNOTENIE POUŽÍVATEĽSKÉHO ROZHRAINIA A RESPONZIVITY

Používateľské rozhranie bolo testované na reprezentatívnej sade zariadení a prehliadačov: mobilné zariadenia (šírka 375 px – iPhone SE, 390 px – iPhone 14), tablety (768 px) a desktopové obrazovky (1280 px a 1920 px).

Tailwind CSS s *mobile-first* prístupom zaručil, že rozhranie sa korektne prispôbuje každej šírke bez manuálneho písania media queries pre každý komponent. Testovacie scenáre zahŕňali:

- Zobrazenie zoznamu inzerátov a detailu na mobilnom zariadení – overenie čitateľnosti textu a dostupnosti tlačidiel.
- Fungovanie interaktívnej mapy (Leaflet) na dotykovom displeji – posúvanie, priblíženie a kliknutie na marker.
- Odoslanie formulára pre pridanie inzerátu na mobile – overenie, že všetky vstupné polia sú použiteľné a validačné správy sú zobrazené správne.

Testovanie neodhalilo žiadne kritické problémy s rozložením ani funkčnosťou. Mierne úpravy boli vykonané pri zobrazení tabuľky v admin paneli na úzkych obrazovkách, kde bol pridaný horizontálny posun (*overflow-x: auto*).

7 DISKUSIA A PRÍNOSY PRÁCE

V tejto kapitole zhodnocujem dosiahnuté výsledky, analyzujem prínosy navrhnutého riešenia a identifikujem oblasti, ktoré ponúkajú priestor pre budúce rozšírenie.

7.1 SPRÁVA OBSAHU BEZ NUTNOSTI REDEPLOYU (SITEMENU, SITECONFIG)

Jedným z najcennejších architektonických rozhodnutí je oddelenie konfigurácie platformy od zdrojového kódu. Bežnou slabinou menších webových projektov je nutnosť nasadiť novú verziu aplikácie vždy, keď treba zmeniť navigačné menu, pridať novú informačnú stránku alebo upraviť globálne nastavenia portálu.

V mojom systéme sú tieto prvky spravované cez dve špeciálne entity:

- **SiteMenu:** Položky navigačného menu sú uložené v databáze s atribútmi `label`, `url`, `order` a `isVisible`. Administrátor môže kedykoľvek pridať, upraviť alebo skryť položku menu bez zásahu do kódu.
- **SiteConfig:** Globálne nastavenia platformy (názov stránky, logo, kontaktný e-mail, odkaz na sociálne siete) sú uložené ako páry kľúč-hodnota v databáze. Frontend načíta tieto hodnoty pri štarte a aplikuje ich naprieč celou platformou.

Výsledkom je *headless CMS* prístup, ktorý dáva administrátorovi plnú kontrolu nad obsahom a konfiguráciou bez technických znalostí. Tento princíp dramaticky znižuje prevádzkové náklady a dobu reakcie na potrebné zmeny v obsahu.

7.2 MOŽNOSTI ĎALŠIEHO ROZŠÍRENIA (PLATOBNÁ BRÁNA, EXTERNÉ ÚLOŽISKO MÉDIÍ)

Navrhnutá architektúra bola od začiatku budovaná s ohľadom na rozšíriteľnosť (Martin 2017). Modulárna štruktúra NestJS umožňuje pridanie nových funkcionálnych celkov bez narušenia existujúceho kódu. Identifikoval som tieto prioritné oblasti rozšírenia:

Platobná brána: Systém obsahuje entitu `Payment`, ktorá je pripravená na integráciu platobnej brány (napr. Stripe alebo Braintree). Implementácia by zahŕňala nový `PaymentsModule` v NestJS, ktorý by spracovával webhooky od platobnej brány a aktualizoval stav platby v databáze. Frontend by doplnil platobný formulár s integráciou príslušného SDK.

Externé úložisko médií: Aktuálna implementácia ukladá obrázky inzerátov ako reťazce kódované vo formáte Base64 priamo do databázy PostgreSQL. Toto riešenie je funkčné pre malý objem dát, ale pri raste platformy by sa odporúčalo presunúť médiá na dedikovanú objektovú úschovňu (napr. AWS S3 alebo Cloudflare R2). NestJS modul by v takom prípade odovzdával príjem súborov balíku `@nestjs/platform-express` s `Multer` middlewarom a ukladal len URL adresu do databázy.

Real-time notifikácie (WebSocket): Rozšírenie komunikačného modulu o WebSocket kanál (NestJS @WebSocketGateway) by umožnilo doručovanie notifikácií o nových správach v reálnom čase bez nutnosti opakovane dopytovať API.

Hodnotenie a recenzie: Pridanie entity Review s prepojením na inzerát a hodnotiteľ a by uzavrelo P2P reputačný kruh, ktorý je základom dôvery v zdieľanej ekonomike (Sundararajan 2014).

8 ZÁVER

8.1 ZHRNUTIE DOSIAHNUTÝCH VÝSLEDKOV

V rámci tejto bakalárskej práce som navrhol a implementoval modernú webovú inzertnú platformu pre oblasť služieb a prenájmu. Výsledkom je funkčný ekosystém pozostávajúci zo štyroch samostatných aplikácií: verejnej platformy pre návštevníkov, používateľskej zóny pre registrovaných členov, administratívneho panela pre správcov a centrálného REST API servera.

Systém je postavený na modernom TypeScript stacku (NestJS, Next.js, PostgreSQL, Prisma ORM) v monorepo architektúre s využitím npm workspaces. Táto voľba umožnila zdieľanie dátových typov a DTO objektov naprieč celým projektom, čo výrazne zvýšilo stabilitu systému a znížilo počet potenciálnych chýb spôsobených nekonzistentnými dátami.

Implementované kľúčové funkcie zahŕňajú: bezpečnú autentifikáciu s JWT tokenmi a bcrypt hashovaním hesiel, systém schvaľovania inzerátov s notifikáciami, interaktívnu mapovú vrstvu s Leaflet, dynamické filtre kategórií s JSONB ukladaním v PostgreSQL, vizuálny editor obsahu GrapesJS a analytický subsystém pre sledovanie správania používateľov.

8.2 NAPLNENIE STANOVENÝCH CIEĽOV

Porovnaním s cieľmi definovanými v úvode práce konštatujem nasledovné:

Cieľ 1 – Stabilné a bezpečné backendové zázemie: Splnený. Backend NestJS aplikácia implementuje JWT autentifikáciu, bcrypt hashovanie, globálnu validáciu vstupov a systém banovania. API je plne zdokumentované cez Swagger.

Cieľ 2 – Tri frontendové aplikácie pre rôzne skupiny používateľov: Splnený. Aplikácie `platform`, `user` a `admin` poskytujú používateľské rozhranie prispôbené každej role. Verejná platforma je optimalizovaná pre SEO, administrácia obsahuje CMS s vizuálnym editorom.

Cieľ 3 – Analytické nástroje a moderácia obsahu: Splnený. Systém asynchrónne zaznamenáva kliknutia na inzeráty a poskytuje administrátorovi grafy návštevnosti a segmentácie. Moduly reportovania a banovania zabezpečujú aktívnu moderáciu komunity.

Platforma tak napĺňa svoju hlavnú misiu: konsolidovať rozptýlenú ponuku lokálnych služieb a prenájmu do jedného transparentného, bezpečného a moderného ekosystému, ktorý odstraňuje nedostatky existujúcich riešení identifikované v analýze súčasného stavu.

POUŽITÁ LITERATÚRA

- Casciaro, M. a L. Mammino (2020). *Node.js Design Patterns. Design and Implement Production-grade Node.js Applications Using Proven Patterns and Techniques*. 3. vyd. Packt Publishing. ISBN: 978-1-839-21411-0.
- Hoffman, A. (2020). *Web Application Security. Exploitation and Countermeasures for Modern Web Applications*. O'Reilly Media. ISBN: 978-1-492-05311-8.
- Cherny, B. (2019). *Programming TypeScript. Making Your JavaScript Applications Scale*. O'Reilly Media. ISBN: 978-1-492-03765-1.
- Jones, M. B., J. Bradley a N. Sakimura (2015). *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/html/rfc7519> (cit. 2025-04-01).
- Martin, R. C. (2017). *Clean Architecture. A Craftsman's Guide to Software Structure and Design*. Prentice Hall. ISBN: 978-0-134-49416-6.
- Obe, R. O. a L. S. Hsu (2017). *PostgreSQL: Up and Running. A Practical Guide to the Advanced Open Source Database*. 3. vyd. O'Reilly Media. ISBN: 978-1-491-96341-8.
- Richardson, L., M. Amundsen a S. Ruby (2013). *RESTful Web APIs. Services for a Changing World*. O'Reilly Media. ISBN: 978-1-449-35806-8.
- Riva, M. (2022). *Real-World Next.js. Build Scalable, High-Performance, and Modern Web Applications Using Next.js, the React Framework for Production*. Packt Publishing. ISBN: 978-1-801-07349-3.
- Rochet, J.-C. a J. Tirole (2003). "Platform Competition in Two-Sided Markets". In: *Journal of the European Economic Association* 1.4, s. 990–1029. DOI: 10.1162/154247603322493212.
- Sommerville, I. (2015). *Software Engineering*. 10. vyd. Pearson. ISBN: 978-0-133-94303-0.
- Sundararajan, A. (2014). "Peer-to-Peer Businesses and the Sharing (Collaborative) Economy: Overview, Economic Effects and Regulatory Considerations". In: *Testimony before the United States House of Representatives Committee on Small Business*. URL: https://smallbusiness.house.gov/uploadedfiles/1-15-2014_revised_sundararajan_testimony.pdf (cit. 2025-04-01).
- Vallas, S. a J. B. Schor (2020). "What Do Platforms Do? Understanding the Gig Economy". In: *Annual Review of Sociology* 46, s. 273–294. DOI: 10.1146/annurev-soc-121919-054857.
- Winters, T., T. Manshreck a H. Wright (2020). *Software Engineering at Google. Lessons Learned from Programming Over Time*. O'Reilly Media. ISBN: 978-1-492-08279-8.